

PPSZ FOR GENERAL K -SAT AND CSP—MAKING HERTLI'S ANALYSIS SIMPLER AND 3-SAT FASTER

DOMINIK SCHEDER AND JOHN STEINBERGER

Abstract.

The currently fastest known algorithm for k -SAT is PPSZ, named after its inventors (Paturi et al. in J ACM 52(3):337-364, 2005. <http://dx.doi.org/10.1145/1066100.1066101>). Analyzing its running time is much easier for input formulas with a unique satisfying assignment. In this paper, we achieve three goals. First, we simplify the analysis of Hertli (in 2011 IEEE 52nd Annual Symposium on Foundations of Computer Science-FOCS 2011, Los Alamitos, 2011) for input formulas with multiple satisfying assignments. Second, we show a “lifting result”: if you improve PPSZ for k -CNF formulas with a unique satisfying assignment, you will immediately get a (weaker) improvement for general k -CNF formulas. In combination this with results by Hansen et al. (in Charikar and Cohen (ed) Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, 2019) and Scheder (in 62nd IEEE Annual Symposium on Foundations of Computer Science, 2021), who all prove improved time bounds for Unique- k -SAT, this gives improved bounds for general k -SAT. We also generalize our results to the domain of Constraint Satisfaction Problems, i.e., satisfiability with more than two truth values.

Keywords.

Boolean satisfiability, Exponential algorithms, Randomized algorithms

Subject classification. 68W20

Contents

1	Introduction	2
1.1	The PPSZ algorithm	5
1.2	Gauging the strength of the proof heuristic P	8
1.3	Previous work	10
1.4	Our contribution	11
1.5	Comparison to conference version	13
1.6	Organization of the paper	14
2	Two failed attempts to prove Main Theorem 1.17	14
2.1	Why direct application of Jensen's inequality will not work	14
2.2	A smarter approach that also fails	16
3	Proof of the Main Theorem 1.17	22
3.1	Unique to general	28
4	Proof of Lemma 3.3	30
5	Constraint satisfaction problems	33
5.1	Proof heuristics for (d, k) -CSPs	35
5.2	A failed attempt at Theorem 5.12	39
5.3	Proof of Theorem 5.12	40
6	Conclusion and open questions	44
	References	45

1. Introduction

The problem of SAT, deciding whether a proposition formula conjunctive normal form has a satisfying assignment (or even constructing such a solution) enjoys a central position among NP-complete problems. The case of k -SAT, in which the input is restricted to k -CNF formulas, i.e., formulas of clause width bounded by k , has drawn particular attention. An obvious brute-force algorithm solves SAT in time $O(2^n \text{poly}(n))$, where n is the number of variables in the input formula. For k -SAT, this running time has

been improved quite a bit. Two approaches stand out: local search algorithms and encoding-based algorithms. [Schöning \(1999\)](#) gave a simple local search algorithm for k -SAT. [Paturi, Pudlák & Zane \(1999\)](#) came up with an encoding-based algorithm, called PPZ in their honor. As an algorithm, PPZ is not as fast as Schöning's, but it has interesting applications in circuit complexity and complexity of exponential algorithms ([Impagliazzo & Paturi 2001](#)). Most importantly for this paper, there exists a “PPZ 2.0 version” named PPSZ after its inventors [Paturi, Pudlák, Saks & Zane \(2005\)](#). This is substantially faster than PPZ and is in fact the currently fastest known algorithm for k -SAT. Although it is very simple to state (which we will do in the next section), it is notoriously challenging to analyze, and its time complexity is still far from being fully understood.

The analysis of PPSZ in [Paturi *et al.* \(2005\)](#) consists of two parts. First, they give an ingenious and elegant analysis of its time complexity on *Unique- k -SAT*, that is, on input formulas with exactly one satisfying assignment. Curiously, this analysis breaks down if the formula has many solutions. Second, the authors give a work-around addressing the *general k -SAT* case, i.e., the case of multiple solutions. Unfortunately, this is quite technical and incurs an exponential loss for $k = 3, 4$. In a breakthrough paper, [Hertli \(2011\)](#) fixed this by coming up with a better (and simpler, yet still quite challenging) analysis. Our work is best seen as a simplification and extension of Hertli's result.

Hertli's result might suggest that formulas with a unique satisfying assignment are indeed the hardest instances—at least for PPSZ. Accordingly, researchers trying to improve the time complexity of k -SAT algorithms have since focused on Unique- k -SAT. A first improvement over PPSZ was achieved by [Hertli \(2014\)](#). He combined PPSZ with several other algorithms and showed that this improves the running time of Unique-3-SAT by a small but exponential amount. His improvement was so tiny that he did not even bother to compute its magnitude. [Qin & Watanabe \(2020\)](#) slightly improved upon it, but like Hertli's result, theirs is tailored toward $k = 3$, with no clear generalization to larger values of k . Only recently, [Hansen, Kaplan, Zamir & Zwick \(2019\)](#) presented

an algorithm called *biased PPSZ* which not only has a “visible” improvement in time complexity but also works for all $k \geq 3$. Scheder (2021) then showed that PPSZ itself, without any change to the algorithm, is exponentially faster than the analysis of Paturi *et al.* (2005) and Hertli (2014) suggest.

When we talk about the time complexity of different k -SAT algorithms, all of which are exponential in n at the time of writing, it is best to ignore polynomial factors (indeed subexponential factors) and write the running time in the form $2^{(1-s_k)n} = 2^n / 2^{s_k n}$ and call s_k the *savings* of the algorithm.¹ The number s_k gauges by how much the algorithm improves over exhaustive search, and it typically depends on k . For example, PPZ has $s_k = 1/k$; Schönning’s random walk algorithm has a time complexity of $\left(\frac{2(k-1)}{k}\right)^n$ which translates to $s_k = \log_2\left(\frac{k}{k-1}\right) = \log(e)/k + o(1/k)$. For both algorithms we have tight examples. For PPSZ, the number s_k is best described as the extinction event in a certain Galton–Watson branching process, but suffice it to say that $s_k \geq \frac{\pi^2}{6k} - o(1/k)$. We should mention that neither Hansen *et al.* (2019) nor Scheder (2021) manage to replace $\frac{\pi^2}{6}$ by a larger constant, i.e., they only manage to improve the $o(1/k)$ part.

We do not know any tight instances for PPSZ. Indeed, Chen, Scheder, Talebanfard & Tang (2013) were the first to show that the worst-case complexity of PPSZ is exponential in n , the number of variables. To be more precise, they construct families $(F_n^k)_{n \geq 0}$ of k -CNF formulas on n variables such that F_n^k has exactly one satisfying assignment and the running time of PPSZ on F_n^k is of the form $2^{(1-\tilde{s}_k)n}$ where $\tilde{s}_k$ go to 0 as k grows. Thus, the improvement over exhaustive search shrinks with growing clause width, which is to say that PPSZ conforms to the *Strong Exponential Time Hypothesis* (Impagliazzo & Paturi 2001). Their construction is based on random sparse systems of linear equations over $GF(2)$. Pudlák, Scheder & Talebanfard (2017) and Scheder & Talebanfard (2020), using a construction based on Tseitin formulas on large-girth graphs, prove stronger lower bounds, showing that the sav-

¹The notation s_k is sometimes used to denote the *supremum* of savings over all k -SAT algorithms; in this paper, s_k means the savings of a particular algorithm.

ings s_k are at most $2/k + o(1/k)$, showing that PPSZ conforms to the *Super Strong Exponential Time Hypothesis* (Vyas & Williams 2019). Note that this sandwiches the true s_k of PPSZ between two reasonably close numbers: $\frac{\pi^2}{6k} - o(1/k) \leq s_k \leq 2/k + o(1/k)$.

1.1. The PPSZ algorithm. PPSZ is a probabilistic algorithm that tries to incrementally construct a satisfying assignment of F . The “generic PPSZ algorithm” is easy to state. Given a k -CNF formula F , choose a variable x therein uniformly at random; then choose a value $b \in \{0, 1\}$. Choose b uniformly at random, unless we can determine the “correct” truth value of x by some correct yet incomplete proof heuristic.

Let us state things more formally. A proof heuristic is a deterministic procedure P which on input F and x outputs a value $b \in \{0, 1, ?\}$. Correctness means that whenever $P(F, x) = b \in \{0, 1\}$ then in fact $F \models (x = b)$, i.e., every satisfying assignment of F sets x to b ; incompleteness means that we allow $P(F, x)$ to output “?”, even if only one value $b \in \{0, 1\}$ for x is feasible. From now on, when we say *proof heuristic*, we always mean a correct but possibly incomplete heuristic. A *partial assignment* β is a function from $V' \subseteq V$ into $\{0, 1\}$. We write $F|_\beta$ to denote the formula arising from F by replacing x by $\beta(x)$ for all x on which β is defined and applying the standard simplification rules—clauses containing a 1 are satisfied and thus removed, constants 0 within a clause are removed.

Suppose now that $\alpha \in \text{sat}(F)$, i.e., it is a satisfying assignment. Below we give procedure ENCODE that, given access to α , F , the heuristic P , and a permutation π of the variables of F , encodes α into a bit string c , hopefully using fewer than n bits. Intuitively, it iterates through the variables in the order given by π and outputs $\alpha(x)$ for every variable, unless this value can already be inferred from F and the bits output so far. This encoding is reversible: the procedure DECODE can recover α when given access to F , P , π , and the encoding c . The generic algorithm RANDOMDECODE then is simply to choose π and c randomly, start decoding and hoping for the best.

Note that the running time of RANDOMDECODE is dominated by the running time of P . Thus, as long as P runs in polynomial

Algorithm 1 Generic Encoding Procedure

```

1: procedure ENCODE( $\alpha, \pi, F, P$ )
2:    $\beta :=$  the empty assignment on  $V$ 
3:   for  $x \in V$  in the order of  $\pi$  do
4:     if  $P(F|_{\beta}, x) = ?$  then
5:       output  $\alpha(x)$ 
6:     end if
7:     add  $[x \mapsto \alpha(x)]$  to  $\beta$ 
8:   end for
9: end procedure

```

Algorithm 2 Generic Decoding Procedure

```

1: procedure DECODE( $c, \pi, F, P$ )
2:    $\beta :=$  the empty assignment on  $V$ 
3:   for  $x \in V$  in the order of  $\pi$  do
4:     if  $P(F|_{\beta}, x) = b \in \{0, 1\}$  then
5:        $\beta(x) := b$ 
6:     else
7:        $\beta(x) :=$  the next bit of  $c$ 
8:     end if
9:   end for
10:  return  $\beta$ 
11: end procedure

```

(subexponential) time, so does RANDOMDECODE. Consequently, we measure the goodness of RANDOMDECODE not in terms of running time, but in terms of *success probability*, which will usually be of the form 2^{-pn} for some constant p . One obtains a Monte Carlo algorithm for k -SAT by repeating RANDOMDECODE for $O(2^{pn})$ times. To make RANDOMDECODE into an algorithm, we still have to specify P . Here are some examples:

EXAMPLE 1.1. The heuristic $P^{(0)}$ that always outputs “?”. Obviously, the algorithm $\text{RANDOMDECODE}(F, P^{(0)})$ is just random guessing, and each solution α appears with probability 2^{-n} . This is not a very good algorithm. $\diamond$

Algorithm 3 Generic Random Decoding Procedure

```

1: procedure RANDOMDECODE( $F, P$ )
2:    $\pi :=$  a random permutation on  $V$ 
3:    $c :=$  a random string in  $\{0, 1\}^n$ 
4:    $\beta :=$  DECODE( $c, \pi, F, P$ )
5:   return  $\beta$  if it satisfies  $F$ , else failure
6: end procedure

```

EXAMPLE 1.2. $P^{(1)}$, the heuristic that answers $P^{(1)}(F, x) = b \in \{0, 1\}$ if F is a CNF formula and F contains the unit clause $(x = b)$; if F contains both $(x = 0)$ and $(x = 1)$ then we say $P^{(1)}(F, x) = 1$ by default, but in this case F is unsatisfiable anyway. $P^{(1)}$ is exactly the heuristic used by PPZ, i.e., $\text{RANDOMDECODE}(F, P^{(1)})$ simply is the algorithm PPZ. Its success probability on k -CNF formulas is $2^{-(1-1/k)n}$. $\diamond$

EXAMPLE 1.3. $P^{(w)}$, a heuristic that generalizes $P^{(1)}$. It answers $P^{(w)}(F, x) = b$ if F is a CNF formula and it contains a subset G of at most w clauses for which $G \models (x = b)$. We can allow w to depend on the number of variables n , and if such $w(n)$ does not grow too fast, then $P^{(w)}$ can be implemented in subexponential time. $\text{RANDOMDECODE}(F, P^{(w)})$ is called *weak* PPSZ. $\diamond$

EXAMPLE 1.4. $\text{BWR}^{(w)}$, standing for *bounded width resolution*. $\text{BWR}^{(w)}(F, x)$ answers b if $(x = b)$ can be derived from F with a resolution derivation of width at most w . Again, w is allowed to grow with n , and as long as $w(n) \in o(n)$, one can implement $\text{BWR}^{(w)}$ in subexponential time. $\text{RANDOMDECODE}(F, \text{BWR}^{(w)})$ is called *strong* PPSZ. $\diamond$

EXAMPLE 1.5. $P^{(\infty)}$. This heuristic employs the whole power of propositional logic. It answers $P^{(\infty)}(F, x) = b \in \{0, 1\}$ if F implies $(x = b)$. Obviously, determining this is itself NP-hard, so this is not an efficient heuristic. Still, it will be important in this paper. Note that if F is satisfiable, then $\text{RANDOMDECODE}(F, P^{(\infty)})$

always outputs a satisfying assignment. Thus, it defines a distribution Q on pairs (π, α) , where π is the permutation it chooses and α the solution it outputs. The distribution Q will play a leading role in our proofs below. $\diamond$

One feature of Hertli (2011) and also of this work is that, in contrast to Paturi *et al.* (2005), Hansen *et al.* (2019), Scheder (2021) and other work, we do not care about the details of the heuristic P but are only interested in one parameter: its error rate, which we will now define.

1.2. Gauging the strength of the proof heuristic P . Let $C_x(\pi, \alpha)$ be the indicator variable which is 1 if ENCODE outputs a bit for x , i.e., if $P(F|_\beta, x) = ?$ in Line 4 of ENCODE. The length of the resulting encoding is $C(\pi, \alpha) := \sum_x C_x(\pi, \alpha)$. Note that $C_x(\pi, \alpha)$ depends on F and P , so we should write $C_x(\pi, \alpha, F, P)$ but since F and P are usually fixed throughout, we choose to drop them for the sake of readability.

OBSERVATION 1.6. *If $\alpha \in \text{sat}(F)$ then*

$$(1.7) \quad \Pr[\text{RANDOMDECODE}(F, P) \text{ returns } \alpha] = \mathbb{E}_\pi [2^{-C(\pi, \alpha)}] ,$$

where $\mathbb{E}_\pi$ is taken with respect to the uniform distribution on permutations.

PROOF. Let $c^* := \text{ENCODE}(\alpha, \pi, F, P)$. RANDOMDECODE returns α iff the first $C(\pi, \alpha)$ bits of its random string $c \in \{0, 1\}^n$ agree with c^* . $\square$

We write $F \models T$ as a shorthand of “ F implies T ”, i.e., every satisfying assignment of F satisfies T . If $F \models (x = 0)$ or $F \models (x = 1)$ we say that x is *frozen in F* . Equivalently, all satisfying assignments of F agree on x . Otherwise, we say that x is *liquid*. Note that $C_x(\pi, \alpha)$ can be 1 for two reasons. First, it could be that in Line 4 of ENCODE, x is liquid in $F|_\beta$ and thus every correct proof heuristic P must answer $P(F|_\beta, x) = ?$. In this case we set $I_x(\pi, \alpha) = 1$. Second, it could be that x is frozen in $F|_\beta$ and $P(F|_\beta, x)$ answers “?” only due to its incompleteness. In this case

we set $J_x(\pi, \alpha) = 1$. Thus, $C_x(\pi, \alpha) = I_x(\pi, \alpha) + J_x(\pi, \alpha)$. We also set $I = \sum_x I_x$ and $J = \sum_x J_x$. Note that $I(\pi, \alpha) = 0$ if F has a unique satisfying assignment, since all variables are frozen. Also, $J(\pi, \alpha) = 0$ for $P^{(\infty)}$, since this heuristic is complete. Here is a plausible notion of strength for proof heuristics: if P is a strong proof heuristic, then the event $[J_x(\pi, \alpha) = 1]$ should not happen too often:

DEFINITION 1.8 (Error of P). *Let $\mathcal{C}$ be a class of formulas and P be a proof heuristic. We say that P has error at most p against $\mathcal{C}$ if for every $F \in \mathcal{C}$, solution α , and variable x in F it holds that $\mathbb{E}_\pi [J_x(\pi, \alpha)] \leq p$.*

THEOREM 1.9 (Paturi *et al.* 1999). $P^{(1)}$ has error $1 - 1/k$ against k -CNF formulas.

Paturi *et al.* (2005) prove the following bound on the error of $P^{(w)}$ (although they do not use this exact wording). Consider the infinite $(k - 1)$ -ary rooted tree. For each vertex v in this tree, choose $\pi_v \in [0, 1]$ uniformly at random. Delete each vertex v with $\pi_v < \pi_{\text{root}}$. Let s_k be probability that the root is contained in a finite connected component, and c_k the probability that said component is infinite. It is easy to see that $c_2 = 0$, and a simple calculation shows that $c_3 = 2 \ln(2) - 1 \approx 0.3863$.

THEOREM 1.10 (Paturi *et al.* 2005). $P^{(w)}$ has error $c_k + \epsilon_{w,k}$ against k -CNF formulas, where $\epsilon_{w,k} \rightarrow 0$ as $w \rightarrow \infty$.

OBSERVATION 1.11. *Let P be a proof heuristic of error at most p against $\mathcal{C}$. If $F \in \mathcal{C}$ has a unique satisfying assignment α , then $\text{RANDOMDECODE}(F, P) = \alpha$ with probability at least 2^{-pn} .*

PROOF. We use [Observation 1.6](#) and Jensen's Inequality:

$$\begin{aligned} \Pr[\text{PPSZ succeeds}] &= \mathbb{E}_\pi [2^{-C(\pi, \alpha)}] \geq 2^{-\mathbb{E}_\pi [C(\pi, \alpha)]} \\ &= 2^{-\mathbb{E}_\pi [J(\pi, \alpha)]} \\ &\geq 2^{-pn} , \end{aligned}$$

where the first inequality is Jensen's inequality, the second equality is because $I(\pi, \alpha) = 0$ due to α being the only satisfying assignment, and the final inequality is because P has error at most p . $\square$

1.3. Previous work. In case F has multiple satisfying assignments, the proof of [Observation 1.11](#) breaks down because I need not be 0, and it is not clear why a proof heuristic of error at most p should give an algorithm of success probability 2^{-pn} . A series of authors have improved PPSZ for the general case of multiple satisfying assignments. [Paturi et al. \(2005\)](#) already gave an analysis, but it incurs exponential loss for $k = 3, 4$. [Iwama & Tamaki \(2004\)](#) combine PPSZ and Schöning's random walk to obtain a better algorithm. This combination was then further explored by [Rolf \(2006\)](#), [Iwama, Seto, Takai & Tamaki \(2010\)](#), and [Hertli, Moser & Scheder \(2011\)](#). All these improvements have serious drawbacks: they still have an exponential loss compared to the Unique- k -SAT bound for $k = 3, 4$; they are extremely technical; they use detailed knowledge of the proof heuristic P ; finally, the latter four have to combine PPSZ with a second algorithm to achieve their improvement. In 2011, [Hertli \(2011\)](#) achieved a breakthrough by proving the following theorem:

HERTLI'S THEOREM 1.12 ([Hertli 2011](#)). *Suppose P has error at most p against $\mathcal{C}$, and $p \geq p^* := \frac{2 - \log(e)}{2} \approx 0.279$. For every satisfiable $F \in \mathcal{C}$, $\text{RANDOMDECODE}(F, P)$ returns a satisfying assignment with probability at least 2^{-pn} .*

Note the mysterious p^* in the theorem. We suspect that it is an artifact of the proof. Anyway, since $c_k \geq p^*$ for all $k \geq 3$, Hertli's theorem works for the current version of PPSZ, for all $k \geq 3$. It might be, however, that future research brings about proof heuristics of error rate less than p^* , in which case the above theorem would again incur an exponential loss. Ingenious as it is, Hertli's proof is quite long and tedious, and offers little intuition as to why it works and why other naive attempts fail.

An obvious question is whether formulas with multiple solutions, now shown not to be harder for PPSZ, are in fact easier, i.e., if F has exponentially many solutions, does PPSZ have exponen-

tially higher success probability? The answer turns out to be yes, at least for $k \geq 5$:

THEOREM 1.13 (Scheder 2019). *Let c_k be the PPSZ time complexity constant defined just before Theorem 1.10. For every $k \geq 5$, there is a strictly increasing function $g_k(\delta)$ with $g_k(0) = 0$ such that if F is a k -CNF formula on n variables with $|\text{sat}(F)| \geq 2^{\delta n}$ then PPSZ on F has success probability at least $2^{-c_k n + g_k(\delta)n - o(n)}$.*

A maybe interesting fact about this result is that it is *not* obtained by extending the analysis of Hertli (2011) or of this work but by going all the way back to the original General- k -SAT analysis in Paturi *et al.* (2005). This is also the reason the theorem fails for $k = 3, 4$.

1.4. Our contribution. The first contribution of this paper is to give a much simpler proof of Hertli’s Theorem 1.12. Our proof in fact highlights why certain naive attempts fail, demonstrates more clearly “what is going on”, and also points toward further improvements.

Second, we generalize the result to (d, k) -CSPs, which are satisfiability problems with more than two truth values.

Finally, we show that any improvement of PPSZ for Unique- k -SAT translates into a (weaker) improvement for General k -SAT, and similarly for CSP. In particular, we will prove a stronger version of Hertli’s Theorem 1.12, which we now explain.

DEFINITION 1.14. *A class $\mathcal{C}$ of formulas or circuits is closed under restrictions if $F \in \mathcal{C}$ implies that $F|_{x=b} \in \mathcal{C}$, for every variable x and value $b \in \{0, 1\}$.*

Note that this applies to most “reasonable” circuit classes, in particular to k -CNF formulas.

DEFINITION 1.15. *We say that a proof heuristic P is monotone if $P(F, x) \in \{0, 1\}$ implies $P(F|_{y=b}, x) \in \{0, 1\}$ for every F , $y \neq x$, and $b \in \{0, 1\}$.*

In other words, if P can deduce the value of x , then it can also do so after we add the additional information that $y = b$. Note that $P^{(0)}, P^{(1)}, P^{(w)}, P^{(\infty)}$ defined above are all monotone.

The procedure $\text{RANDOMDECODE}(F, P^{(\infty)})$ chooses a uniformly random permutation $\pi \in \text{Sym}(V)$ and always outputs a satisfying assignment. Thus, it defines a distribution Q on $\text{Sym}(V) \times \text{sat}(F)$.

OBSERVATION 1.16. $Q(\pi, \alpha) = \frac{1}{n!} \cdot 2^{-I(\pi, \alpha)}$.

PROOF. The probability that RANDOMDECODE chooses a particular permutation π is $\frac{1}{n!}$. Once this has been chosen, it will return α iff the first $C(\pi, \alpha)$ bits of the random string c agree with $c^* := \text{ENCODE}(\alpha, \pi, F, P)$. Thus, the probability that it chooses π and outputs α is $\frac{1}{n!} 2^{-C(\pi, \alpha)}$. Now observe that $P^{(\infty)}$ is complete, thus $J(\pi, \alpha) = 0$ and $C(\pi, \alpha) = I(\pi, \alpha)$. $\square$

MAIN THEOREM 1.17. *Suppose P has error at most p against $\mathcal{C}$ for some $p \geq p^* := \frac{2 - \log(e)}{2} \approx 0.279$ and set $q := p - p^* \geq 0$. If $F \in \mathcal{C}$ is satisfiable then RANDOMDECODE returns a satisfying assignment with probability at least $2^{-pn+q \mathbb{E}_{(\pi, \alpha) \sim Q}[I(\pi, \alpha)]}$.*

Since $q \geq 0$ the probability is at least 2^{-pn} , which immediately implies [Hertli's Theorem 1.12](#). Our theorem has a nice by-product, a “lifting result” from Unique- k -SAT to General k -SAT: suppose you have an algorithm A which is exponentially better than PPSZ for Unique- k -SAT. Given an input k -CNF formula F , there are two cases: first, it could be that $\mathbb{E}_Q[I] \geq \delta n$ for some fixed δ , in which case the bound in [Main Theorem 1.17](#) is already exponentially larger than 2^{-pn} ; or $\mathbb{E}_Q[I] < \delta n$, in which case there is a restriction ρ such that $F|_\rho$ has a unique satisfying assignment and ρ sets fewer than δn variables. We can now guess ρ and apply A to $F|_\rho$. If we choose δ small enough, then the probability of guessing ρ correctly is large enough to not offset the benefit of A , and the overall success probability is still exponentially better than PPSZ. Formally, we obtain the following theorem:

LIFTING THEOREM 1.18. *Suppose P is a monotone proof heuristic with error rate at most p against class $\mathcal{C}$. We assume that $\mathcal{C}$ is closed under restrictions and that $p > p^*$.*

- (i) *If procedure RANDOMDECODE solves UNIQUE- $\mathcal{C}$ -SAT with success probability at least $2^{(-p+\epsilon)n}$, then it solves $\mathcal{C}$ -SAT with success probability $2^{(-p+\epsilon')n}$.*
- (ii) *If there is a probabilistic algorithm A for UNIQUE- $\mathcal{C}$ -SAT with success probability $2^{(-p+\epsilon)n}$, then there is a probabilistic algorithm A' for $\mathcal{C}$ -SAT with success probability at least $2^{(-p+\epsilon')n}$ and running time n times that of A .*
- (iii) *If there is Monte Carlo algorithm B solving UNIQUE- $\mathcal{C}$ -SAT running in time $2^{(p-\epsilon)n}$, then there exists a Monte Carlo algorithm B' solving $\mathcal{C}$ -SAT in time $2^{(p-\epsilon')n}$.*

In all three cases, $\epsilon' > 0$ if $\epsilon > 0$, and in particular $\epsilon' \geq C \cdot \frac{\epsilon}{\log 1/\epsilon}$ for some C depending on $p - p^$.*

Using (ii), Hansen *et al.* (2019) could immediately translate their analysis of biased PPSZ for Unique- k -SAT to the general case:

THEOREM 1.19 (Hansen *et al.* 2019). *The probabilistic algorithm Biased PPSZ, which has subexponential running time, has success probability at least $2^{(-c_k+\epsilon_k)n}$ on k -SAT, where $\epsilon_k > 0$ for all $k \geq 3$.*

Similarly, Scheder (2021) used Lifting Theorem 1.18(i) to argue that PPSZ itself has success probability exponentially greater than $2^{-c_k n}$ on general k -CNF formulas:

THEOREM 1.20 (Scheder 2021). *The success probability of PPSZ on k -CNF formulas is at least $2^{(-c_k+\epsilon_k)n}$ for some $\epsilon_k > 0$.*

1.5. Comparison to conference version. An extended abstract of this work appeared at CCC 2017 (Scheder & Steinberger 2017). This version differs from it by including all proofs and extending the result to (d, k) -CSP problems, i.e., satisfiability problems in which the variables can take on more than two values.

We also include several examples that serve as a warning for those who strive to find a simpler proof. Indeed, analyzing PPSZ on formulas with multiple satisfying assignments is subtle, and we have personally encountered several instances where people came up with erroneous proofs. Thus, if the reader thinks they have found an even simpler proof, they should not be discouraged, but they should at least run their proof against the examples we provide in this work.

1.6. Organization of the paper. We start with two failed attempts at proving the [Main Theorem 1.17](#). We illustrate the failure by giving a concrete CNF formula on which the proposed analysis gives a time bound that is exponentially worse than what [Main Theorem 1.17](#) claims. In particular the second attempt and its counter-example are quite interesting. We encourage the reader to read through it—although it is quite lengthy, the idea used later on in the “real” proof is developed within this example.

In [Section 3](#) we prove the [Main Theorem 1.17](#) but postpone the proof of the key lemma to [Section 4](#). [Section 5](#) generalizes the proof to CSPs on more than two colors.

2. Two failed attempts to prove [Main Theorem 1.17](#)

Before giving the real proof, we will show two attempts that fail and thus illustrate that proving [Hertli’s Theorem 1.12](#) requires non-trivial effort.

2.1. Why direct application of Jensen’s inequality will not work. Let us proceed as in the proof of [Observation 1.11](#):

$$\begin{aligned} \Pr_{c,\pi}[\text{DECODE}(c, \pi, F, P) = \alpha] &= \mathbb{E}_{\pi} [2^{-C(\pi, \alpha)}] \\ &\geq 2^{-\mathbb{E}_{\pi}[C(\pi, \alpha)]}, \end{aligned}$$

where the inequality follows from Jensen’s inequality applied to the convex function $t \mapsto 2^{-t}$. If α is the unique satisfying assignment, we are done. Indeed, in that case $I(\pi, \alpha) = 0$ and thus

$C(\pi, \alpha) = J(\pi, \alpha)$ and we conclude that $\mathbb{E}[C(\pi, a)] = \mathbb{E}[J(\pi, a)] = \sum_x \mathbb{E}[J_x(\pi, a)] \leq pn$ because P has error rate at most p .

If F has multiple satisfying assignments, we have to sum the above expression over the whole set $\text{sat}(F)$. The success probability of RANDOMDECODE is

$$\begin{aligned} \Pr_{c, \pi}[\text{success}] &= \sum_{\alpha \in \text{sat}(F)} \Pr_{c, \pi}[\text{DECODE}(c, \pi, F, P) = \alpha] \\ (2.1) \quad &= \sum_{\alpha \in \text{sat}(F)} \mathbb{E}_{\pi} [2^{-C(\pi, \alpha)}] \end{aligned}$$

$$(2.2) \quad \geq \sum_{\alpha \in \text{sat}(F)} 2^{-\mathbb{E}_{\pi}[C(\pi, \alpha)]} ,$$

We will construct an example in which $\Pr[\text{success}] = 1$ but (2.2) is exponentially small. Consider $P = P^{(\infty)}$, the complete proof heuristic, which has error 0 against all CNF formulas. In this case (2.1) is 1, as DECODE always returns a satisfying assignment if given access to $P^{(\infty)}$. Let F be the Boolean function defined by $F(x) = 1$ if $|x| = 1$, i.e., exactly one of the n positions of x is 1, so $\text{sat}(F) = \{\mathbf{e}_1, \dots, \mathbf{e}_n\}$; or rather let F be some CNF formula representing this Boolean function. Note that since $P^{(\infty)}$ is the complete prover, it does not really matter in which way we represent F .

By symmetry, $\Pr[\text{DECODE}(c, \pi, F, P^{(\infty)}) = \mathbf{e}_i] = 1/n$ for every i . What is $C(\pi, \mathbf{e}_1)$? Once ENCODE($\mathbf{e}_1, \pi, F, P^{(\infty)}$) has output the value of x_1 under $\mathbf{e}_1$, which is 1, our heuristic $P^{(\infty)}$ “knows” that all remaining bits must be 0; thus, x_1 will be the last variable whose value is encoded. Also note that the variables coming before x_1 in π , all of which have value 0 in $\mathbf{e}_1$, are liquid, and thus, ENCODE must output a value for them. We conclude that ENCODE outputs the values for all variables up to x_1 , including x_1 , unless x_1 comes at the very end, in which case we have already output $n - 1$ many 0’s and $P^{(\infty)}$ knows that the value of x_1 must be 1. With j denoting the position of x_1 in π , we conclude that $C(\pi, \mathbf{e}_i) = \min(j, n - 1)$ and thus

$$\mathbb{E}_{\pi}[C(\pi, \mathbf{e}_1)] = \frac{1}{n} \cdot \sum_{j=1}^{n-1} j + \frac{1}{n}(n - 1) \geq \frac{\binom{n}{2}}{n} = \frac{n-1}{2} .$$

The same bound holds for each $\mathbf{e}_i$. Summing up over $\text{sat}(F)$ gives

$$(2.2) \quad = \sum_{\alpha \in \text{sat}(F)} 2^{-\mathbb{E}_{\pi}[C(\pi, \alpha)]} \leq n \cdot 2^{-\frac{n-1}{2}}.$$

Thus, there is an exponential gap between (2.2) and $2^{-pn} = 1$.

What went wrong? Jensen's inequality, here used in the form $\mathbb{E}[2^{-X}] \geq 2^{-\mathbb{E}[X]}$ is tight if the random variable X is a constant, and is almost tight if X is highly concentrated. Note that in the previous example, the random variable $C(\mathbf{e}_1, \pi)$ is far from concentrated: it takes on the values $1, 2, 3, \dots, n-2$ with probability $1/n$ each and $n-1$ with probability $2/n$. So maybe we should try to replace $C(\pi, \alpha)$ by some random variable with better concentration properties.

2.2. A smarter approach that also fails. If we run the procedure DECODE (Algorithm 2) with random c and random π , there are two cases in each step. Either x is liquid, in which case we cannot make a mistake. Or x is frozen. But if x is frozen, don't we already know that, with probability at least $1-p$ our heuristic P can infer the correct value? If this reasoning holds water, proving Hertli's Theorem 1.12 should be straightforward.

In a certain way, our approach in this section can be seen as an attempt to make the above reasoning rigorous. Recall the distribution $Q(\pi, \alpha)$ on $\text{Sym}(V) \times \text{sat}(F)$: we imagine that we run $\text{DECODE}(c, \pi, F)$ with random c and π and the complete prover $P^{(\infty)}$. Since this always returns a satisfying assignment, it defines a probability distribution Q over $\text{Sym}(V) \times \text{sat}(F)$. By Observation 1.16 we know that

$$Q(\pi, \alpha) = \frac{1}{n!} \cdot 2^{-I(\pi, \alpha)}.$$

We can now rewrite the success probability of DECODE (using some incomplete proof heuristic P) as

$$(2.3) \quad \begin{aligned} \Pr[\text{success}] &= \sum_{\alpha \in \text{sat}(F)} \mathbb{E}_{\pi} [2^{-C(\pi, \alpha)}] = \sum_{\pi, \alpha} \frac{1}{n!} 2^{-I(\pi, \alpha) - J(\pi, \alpha)} \\ &= \mathbb{E}_{(\pi, \alpha) \sim Q} [2^{-J}] \end{aligned}$$

$$(2.4) \quad \geq 2^{-\mathbb{E}_Q[J]}.$$

Since this was just symbolic manipulation of an expectation, let us add a bit of intuition to (2.4). In a thought experiment, we run `RANDOMDECODE` with both $P^{(\infty)}$ and P . We make our choices according to $P^{(\infty)}$, but whenever P answers “?” but $P^{(\infty)}$ answers a value in $\{0, 1\}$ we have to pay a coin. Each coin represents an event $[J_x(\pi, \alpha) = 1]$ and thus an opportunity of P to make a fatal mistake. The total number of coins paid is J . So we should bound $\mathbb{E}_Q[J]$, the expected numbers of coins.

Unfortunately, there is an example where we pay much more than pn coins on expectation, and (2.4) is exponentially smaller than 2^{-pn} . Consider the following function:

$$\text{EXACTLY-TWO}(x, y, z) \wedge \bigwedge_{i=1}^n (\text{AT-LEAST-TWO}(x, y, z) \rightarrow a_i)$$

We can express this as a 3-CNF formula F :

$$(x \vee y) \wedge (x \vee z) \wedge (y \vee z) \wedge (\bar{x} \vee \bar{y} \vee \bar{z}) \wedge \bigwedge_{i=1}^n ((\bar{x} \vee \bar{y} \vee a_i) \wedge (\bar{x} \vee \bar{z} \vee a_i) \wedge (\bar{y} \vee \bar{z} \vee a_i)) .$$

Enumerating our variables as $x, y, z, a_1, \dots, a_n$, the satisfying assignments are $\alpha_1 = (0111^n)$, $\alpha_2 = (1011^n)$, and $\alpha_3 = (1101^n)$. Consider the prover $P = P^{(1)}$, i.e., it checks whether the variable in question is contained in a unit clause. Since this is a 3-CNF, the error rate of P is at most $2/3$ by Theorem 1.9. What is $\mathbb{E}_Q[J]$? By symmetry between the α_i , we see that

$$\begin{aligned} \mathbb{E}_Q[J] &= \mathbb{E}_{\alpha \sim Q} [\mathbb{E}_{\pi \sim Q|\alpha} [J]] = \mathbb{E}_{\pi \sim Q|\alpha_1} [J(\alpha_1, \pi)] \\ &\geq \sum_{i=1}^n \mathbb{E}_{\pi \sim Q|\alpha_1} [J_{a_i}(\alpha_1, \pi)] \\ &= n \mathbb{E}_{\pi \sim Q|\alpha_1} [J_{a_1}(\alpha_1, \pi)] , \end{aligned}$$

where the last equality follows from the symmetry between the a_i . What is $\mathbb{E}_{\pi \sim Q|\alpha_1} [J_{a_1}(\pi, \alpha_1)]$? This is a slightly weird expression. Note that

$$Q(\pi, \alpha) = Q(\pi)Q(\alpha|\pi) = \frac{1}{n!} 2^{-I(\pi, \alpha)}$$

is the “natural” order of conditioning, which follows directly from the way we defined Q . In contrast, going the other way,

$$Q(\pi, \alpha) = Q(\alpha)Q(\pi|\alpha)$$

does not really have any intuitive interpretation. Still, it is of course completely valid mathematically, and we can compute the expectation of J_{a_1} with respect to Q conditioned on $\alpha = \alpha_1$, and see that $\mathbb{E}_{\pi \sim Q|\alpha_1}[J_{a_1}(\pi, \alpha_1)] = \frac{11}{16} > 2/3$. Thus, $2^{-E_Q[J]} \leq 2^{-\frac{11}{16}n}$, which is exponentially smaller than $2^{-\frac{2}{3}n}$. The latter is true worst-case success probability of PPZ (i.e., PPSZ with proof heuristic $P^{(1)}$) on 3-CNF formulas.

We will now walk the reader through a complete and detailed analysis of this example, in particular computing the quantity

$$(2.5) \quad \mathbb{E}_{\pi \sim Q|\alpha_1}[J_{a_1}(\pi, \alpha_1)] .$$

Although not strictly necessary, working through this example will sketch out the key ideas we need to achieve an actual proof. We will first compute (2.5) directly, from the definition of conditional expectations; in a second run, we will try to compute it from a less rigorous but more illustrative way.

Rigorous analysis of the example. Recall the satisfying assignment $\alpha_1 = (0111^n)$. It sets x to 0 and $y, z, a_1, \dots, a_n$ to 1. From the definitions, we get

$$(2.5) = \frac{1}{Q(\alpha_1)} \mathbb{E}_{(\pi, \alpha) \sim Q} [\mathbf{1}_{\alpha=\alpha_1} J_{a_1}(\pi, \alpha)] .$$

By symmetry, the marginal probability $Q(\alpha_1) = \frac{1}{3}$. For a permutation π on V and a subset W , let $\pi|_W$ be the permutation on W induced by π ; that is, we obtain $\pi|_W$ by ordering the elements of V according to π and then removing all elements in $V \setminus W$. Suppose π' and π'' are permutations on V with $\pi'|_{\{x,y,z\}} = \pi''|_{\{x,y,z\}}$. Then $I(\pi', \alpha) = I(\pi'', \alpha)$. The reason is that all a_i are already frozen to 1 in F , so encoding $a_i = 1$ does not add any further information. Consequently, $Q(\pi', \alpha) = Q(\pi'', \alpha)$. In our case, this means that we can determine $Q(\pi, \alpha_1)$ just by looking at the order in which

x, y, z appear in π . Furthermore, this means that after revealing the order of x, y, z in π , the position of a_1 relative to x, y, z is still uniform among the four possibilities. In the following table we let τ denote the concrete value of $\pi|_{\{x,y,z\}}$.

τ	$I(\pi, \alpha_1)$	$Q(\tau, \alpha \mid \alpha = \alpha_1)$	$\mathbb{E}[J_{a_1}(\pi, \alpha) \mid \pi_{\{x,y,z\}} = \tau, \alpha = \alpha_1]$
xyz	1	$\frac{1}{6} \cdot \frac{1}{2}$	$\frac{3}{4}$
xzy	1	$\frac{1}{6} \cdot \frac{1}{2}$	$\frac{3}{4}$
yxz	2	$\frac{1}{6} \cdot \frac{1}{4}$	$\frac{3}{4}$
zxy	2	$\frac{1}{6} \cdot \frac{1}{4}$	$\frac{3}{4}$
yzx	2	$\frac{1}{6} \cdot \frac{1}{4}$	$\frac{2}{4}$
zyx	2	$\frac{1}{6} \cdot \frac{1}{4}$	$\frac{2}{4}$

To understand this table, look for example at the third row, where τ is yxz . We know that $\alpha_1(x) = 0$ and $\alpha_1(y) = 1$. In the beginning, y is liquid, so $I_y = 1$; after fixing $y = 1$, note that x is still liquid—we could be in α_1 or in α_3 , thus $I_x = 1$, too; finally, when fixing $y = 1$ and $x = 0$, the variable z becomes frozen (in fact α_1 is now the only possible solution) and $I_z = 0$, and of course $I_{a_i} = 0$ since the a_i are frozen to begin with. Thus, $I(\pi, \alpha_1) = 2$ when $\pi|_{\{x,y,z\}} = yxz$. Consequently, $Q(\tau, \alpha_1) = \frac{1}{6} \cdot \frac{1}{4}$. The $\frac{1}{6}$ simply represents the uniform distribution on $\text{Sym}(\{x, y, z\})$. If we know that yxz appear in that order, what is $\mathbb{E}[J_{a_1}(\pi, \alpha_1)]$? Looking at the 3-CNF formula F above, we see that $J_{a_1}(\pi, \alpha_1)$ is 1 unless a_1 comes after y and z (and thus can be inferred by $P^{(1)}$). Among the four positions a_1 can have relative to y, x, z , there is only coming after both y and z —the very last one. Therefore, $\mathbb{E}[J_{a_1}] = \frac{3}{4}$. We obtain the other rows of the table by similar reasoning.

Next, if we had a uniform distribution over the τ , each having probability $\frac{1}{6}$, then $\mathbb{E}[J_{a_1}]$ would be

$$\frac{1}{6} \cdot \left(\frac{3}{4} + \frac{3}{4} + \frac{3}{4} + \frac{3}{4} + \frac{2}{4} + \frac{2}{4} \right) = \frac{2}{3},$$

which is exactly the error rate of $P^{(1)}$ against 3-CNF formulas. Alas, conditioning on α_1 makes π , and thus τ , non-uniformly distributed. Note that the probabilities $Q(\tau, \alpha_1)$ in the above table

sum up to $Q(\alpha_1) = \frac{1}{3}$; thus, to compute the conditional expectation, we have to additionally multiply the probabilities by 3. Realizing that the rows appear in pairs, we see that the conditional expectation is

$$3 \cdot 2 \cdot \left(\frac{1}{6} \cdot \frac{1}{2} \cdot \frac{3}{4} + \frac{1}{6} \cdot \frac{1}{4} \cdot \frac{3}{4} + \frac{1}{6} \cdot \frac{1}{4} \cdot \frac{2}{4} \right) = \frac{11}{16}.$$

This was the rigorous but somewhat mechanical computation, which hopefully convinced the reader that (2.5) indeed equals $\frac{11}{16}$.

More intuitive analysis of the example. Next, we will give a handwaving but more useful analysis of the same example. Instead of computing the conditional distribution of π , or of $\pi|_{\{x,y,z\}}$, we just focus on which of x, y, z, a_1 comes first under π . Under a uniform distribution, each should be first with probability $\frac{1}{4}$. To analyze how this changes when conditioning on $\alpha = \alpha_1$, note that $Q(\pi|\alpha_1)$ is proportional to $Q(\pi, \alpha_1)$. As already argued above, moving a_1 around in π will not change $Q(\pi, \alpha_1)$, and thus, the probability of a_1 coming first among x, y, z, a_1 is still $\frac{1}{4}$. What about x ? If x comes first then setting $x = 0$ freezes y to 1 and $z = 1$ and leaves α_1 as the only possible assignment. However, if x comes first then setting $x = 1$ leads to some $\alpha \neq \alpha_1$. Both happen with equal probability. We observe that when x comes first, then α_1 will be sampled with probability $\frac{1}{2}$, whereas the a priori probability is $\frac{1}{3}$. In other words, x being first increases the probability of α_1 by a factor of $\frac{3}{2}$. Thus, the conditional probability of x coming first among x, y, z , should also be a $\frac{3}{2}$ factor larger than the uniform $\frac{1}{3}$ —it should be $\frac{1}{2}$. A quick glance at the table above shows that this is indeed the case. If x comes first among x, y, z , the probability that a_1 comes before x is $\frac{1}{4}$, and therefore, x comes first among x, y, z, a_1 with probability $\frac{1}{2} \cdot \frac{3}{4} = \frac{3}{8}$. This leaves $1 - \frac{1}{4} - \frac{3}{8} = \frac{3}{8}$ for y or z coming first; thus, by symmetry each of y and z comes first among x, y, z, a_1 with probability $\frac{3}{16}$.

variable	x	y	z	a_1
Pr[first]	$\frac{3}{8}$	$\frac{3}{16}$	$\frac{3}{16}$	$\frac{1}{4}$

In a nutshell, conditioning on α_1 pulls x to the front of π and automatically pushes some other variables to the back y and z . In

this example we are unfortunate that those variables— y and z —are exactly those whose value $P^{(1)}$ would need to infer a_1 . Pulling x to the front makes $P^{(1)}$ less likely to infer a_1 .

Taking yet another step back, the problem is not really that x gets pulled to the front—it is that y and z each are less likely to be selected next than a_1 .

Intuition. The problem is that some liquid variables (here: y or z) are less likely to be selected next in π than a particular frozen variable (here: a_1).

Magnitude of the problem. We have found the core issue— a_1 is more likely to come next than y , and this adversely affects J_{a_1} . Let us gauge how large this effect can be. As seen above, the key quantity is the factor by which the probability of a_1 changes when a particular variable comes first. Moving away from our particular formula F , can we give some upper bound on how much x coming next in π can *increase* α_1 ? Not really. A priori, α_1 can be arbitrarily unlikely; it is easy to construct a formula where α_1 is the only solution with $\alpha_1(x) = 0$, and thus, putting x first suddenly increases the probability of α_1 to $\frac{1}{2}$ —if we set x to 0, then α_1 remains the only solution. Thus, conditioning on α_1 might arbitrarily *increase* the probability of some liquid variable x to be first in π . But as argued above, this is not really a problem. By which factor can selecting some other liquid variable y *decrease* α_1 ? Suppose ρ is the a priori probability of α_1 under Q . If we select y first and set it to $\neg\alpha_1(y)$, that probability drops from ρ to 0. However, if we set it to $\alpha_1(y)$, the probability of α_1 cannot go down. We see that putting some liquid y first in π can decrease the probability of α_1 by at most a factor of 2.

What if we replaced Q by some other distribution R under which each liquid variable is *twice* as likely to come first in π as each frozen variable? If our reasoning above holds water, then conditioning on some particular α_1 can at most halve the probability of a liquid y coming first, so even after conditioning, each liquid variable is *at least as likely* to be first as each frozen variable, and thus, our proof heuristic P should still work as promised.

This opens two questions. (1) Can define such an R ? (2) Can we arbitrarily change distribution Q to R ? After all, Q appeared pretty naturally in (2.3), and it is not clear whether we can obtain a similar expression involving R . The answer to both questions is yes. Achieving (1) is easy; (2) is not difficult either but feels somewhat unnatural and in fact is the step that leads to the mysterious constant $p^* = \frac{2 - \log(e)}{2} \approx 0.279$ in [Hertli's Theorem 1.12](#) and [Main Theorem 1.17](#).

3. Proof of the [Main Theorem 1.17](#)

In addition to $Q(\pi, \alpha) = \frac{1}{n!} \cdot 2^{-I(\pi, \alpha)}$, we consider another distribution R on $\text{Sym}(V) \times \text{sat}(F)$, which we mentioned in the last two paragraphs and which we will define shortly. We estimate the success probability of `RANDOMDECODE`:

$$\begin{aligned} \Pr[\text{success}] &= \sum_{\alpha \in \text{sat}(F)} \mathbb{E}_{\pi} [2^{-C(\pi, \alpha)}] = \sum_{\alpha \in \text{sat}(F)} \frac{1}{n!} \sum_{\pi} 2^{-I(\pi, \alpha) - J(\pi, \alpha)} \\ &= \sum_{\alpha \in \text{sat}(F)} \sum_{\pi} R(\pi, \alpha) \frac{2^{-I(\pi, \alpha) - J(\pi, \alpha)}}{n! R(\pi, \alpha)} \\ &= \mathbb{E}_{(\pi, \alpha) \sim R} \left[\frac{Q(\pi, \alpha)}{R(\pi, \alpha)} \cdot 2^{-J(\pi, \alpha)} \right] \end{aligned}$$

(by Jensen's inequality)

$$\begin{aligned} &\geq 2^{\mathbb{E}_R[-\log_2(\frac{R(\pi, \alpha)}{Q(\pi, \alpha)}) - J(\pi, \alpha)]} \\ &= 2^{-\text{KL}(R||Q) - \mathbb{E}_R[J(\pi, \alpha)]}, \end{aligned}$$

where $\text{KL}(R||Q)$ is called the *Kullback–Leibler divergence from Q to R* which, roughly speaking, measures how different R is from Q —although it fails to be a *metric* on probability distributions. The definition of $\text{KL}(R||Q)$ for probability distributions R, Q over a finite set Ω is defined to be

$$\text{KL}(R||Q) := \sum_{\omega \in \Omega} R(\omega) \log_2 \left(\frac{R(\omega)}{Q(\omega)} \right).$$

We can now plug in any distribution R and aim to minimize the expression

$$(3.1) \quad \text{KL}(R||Q) + \mathbb{E}_{(\pi, \alpha) \sim R} [J(\pi, \alpha)] .$$

Here we face a tradeoff. If we choose R to be the uniform distribution over $\text{Sym}(V) \times \text{sat}(F)$, we get

$$\mathbb{E}_R[J(\pi, \alpha)] = \mathbb{E}_\alpha \left[\sum_x \mathbb{E}_\pi [J_x(\pi, \alpha)] \right] \leq pn ,$$

since P has error at most p ; however, $\text{KL}(R||Q)$ might be too large. Choosing $R = Q$ makes $\text{KL}(R||Q) = 0$, but the second term can become larger than pn , as we have just seen in the previous section.

Remember our goal, stated toward the end of the previous section: define a distribution R under which each liquid variable is *twice* as likely to come first in $\pi \sim R$ than each frozen variable. Stated differently, the probability of x being chosen should be proportional to the number of $b \in \{0, 1\}$ for which $F|_{x=b}$ is satisfiable. We achieve this by choosing a “one-variable assignment” $x \mapsto b$ from all such assignments that keep the formula satisfiable. Algorithm 4 makes this formal.

Algorithm 4 Sampling from the distribution R

```

1: procedure SAMPLE-R( $F, V$ )
2:   if  $V = \emptyset$  then
3:     return  $(\emptyset, \emptyset)$ 
4:   end if
5:    $S(F) := \{(x, b) \in V \times \{0, 1\} \mid F|_{x=b} \text{ is satisfiable} \}$ 
6:    $(x, b) :=$  a random element from  $S$ 
7:    $(\pi, \alpha) := \text{SAMPLE-R}(F|_{x=b}, V \setminus \{x\})$ 
8:   return  $(x\pi, \alpha \cup [x = b])$ 
9: end procedure

```

The process SAMPLE-R in Algorithm 4 defines a distribution R on pairs (π, α) that resembles Q (and thus keeps the divergence $\text{KL}(R||Q)$ small) while showing a moderate preference for moving frozen variables to the back of π (keeping $\mathbb{E}_R[J(\pi, \alpha)]$ small).

Note that unlike under Q , the marginal distribution on permutations induced by R is not necessarily uniform. Indeed, if we call $\text{SAMPLE-R}(F, V)$ for $F = x$ and $V = \{x, y\}$ then π is (y, x) with probability $2/3$ and (x, y) with probability $1/3$. On the other hand, R and Q induce the same marginal distribution on satisfying assignments. The reader is encouraged to verify this, but this property is not required for the proof. We call the resulting distribution R_F to highlight its dependency on F . If F is understood from the context, we simply write R .

LEMMA 3.2. $\text{KL}(R||Q) \leq p^* \mathbb{E}_R[I]$ for every F .

PROOF. Let us spell out a pair (π, α) as $(x_1 \dots x_n, b_1 \dots b_n)$, where x_i is the i^{th} variable under π and $b_i = \alpha(x_i)$. Let $\tau_i := (x_1 \dots x_i, b_1 \dots b_i)$ be a “prefix” of (π, α) . Define R_{τ_i} be the distribution of (b_{i+1}, x_{i+1}) under R conditioned on τ_i . Similarly, define Q_{τ_i} . By the chain rule for the divergence, we get

$$\text{KL}(R||Q) = \sum_{i=0}^{n-1} \mathbb{E}_{\tau_i \sim R} [\text{KL}(R_{\tau_i}||Q_{\tau_i})].$$

So let us fix a “past” τ_i and bound $\text{KL}(R_{\tau_i}||Q_{\tau_i})$. Let $F_i := F|_{x_1 \mapsto b_1 \dots x_i \mapsto b_i}$ and $V_i := \{x_{i+1}, \dots, x_n\}$. So F_i is a CNF formula over V_i , and it is exactly the formula for which SAMPLE-R is called in its i^{th} call. Let $n_i = |V_i|$, $s_i := |S(F_i, V_i)|$, f_i the number of frozen variables in V_i and l_i the number of liquid variables. Thus, $f_i + l_i = n_i$ and $f_i + 2l_i = s_i$. Note that R_{τ_i} is uniform over $S(F_i, V_i)$. Q_{τ_i} picks x_{i+1} uniformly at random from V_i and assigns it a random value from the (one or two) allowed values. Thus, $Q_{\tau_i}(x, b)$ is 0 if $(x, b) \notin S(F_i, V_i)$; otherwise, it is $1/n_i$ if x is frozen and $1/2n_i$ if x is liquid. We can now compute:

$$\begin{aligned} \text{KL}(R_{\tau_i}||Q_{\tau_i}) &= \sum_{(x,b) \in S(F_i, V_i)} R_{\tau_i}(x, b) \log \left(\frac{R_{\tau_i}(x, b)}{Q_{\tau_i}(x, b)} \right) \\ &= \sum_{(x,b) \in S(F_i, V_i)} \frac{1}{s_i} \log \left(\frac{1/s_i}{\begin{bmatrix} 1/n_i & \text{if } x \text{ frozen,} \\ 1/2n_i & \text{if } x \text{ liquid} \end{bmatrix}} \right). \end{aligned}$$

The expression in the parenthesis is unwieldy, but we know exactly how often each of two cases occurs: f_i and l_i times, respectively.

Therefore,

$$\begin{aligned}
 \text{KL}(R_{\tau_i} || Q_{\tau_i}) &= \frac{2l_i}{s_i} \log \left(\frac{1/s_i}{1/2n_i} \right) + \frac{f_i}{s_i} \log \left(\frac{1/s_i}{1/n_i} \right) \\
 &= \frac{2l_i}{s} \log \left(\frac{2n_i}{s_i} \right) + \frac{f_i}{s_i} \log \left(\frac{n_i}{s_i} \right) \\
 &= \frac{2l_i}{s} + \log \left(\frac{n_i}{s_i} \right) \\
 &= \frac{2l_i}{s} + \log \left(1 - \frac{l_i}{s_i} \right) \\
 &< \frac{2l_i}{s} - \log(e) \frac{l_i}{s_i} = \frac{l_i}{s_i} (2 - \log(e)) .
 \end{aligned}$$

Let $\tilde{I}_i(\pi, \alpha) := I_{x_i}(\pi, \alpha)$, i.e., an indicator variable which is 1 if the i^{th} variable under π is liquid in F_{i-1} . We observe that $\mathbb{E}_{R_{\tau_i}}[\tilde{I}_{i+1}] = \frac{2l_i}{s_i}$, since there are exactly $2l_i$ pairs $(x, b) \in S(F_i, V_i)$ for which the variable x is liquid. Putting everything together, we get

$$\begin{aligned}
 &\text{KL}(R || Q) \\
 &\leq \sum_{i=0}^{n-1} \mathbb{E}_{\tau_i \sim R} \left[\frac{l_i}{s_i} (2 - \log(e)) \right] = \frac{2 - \log(e)}{2} \sum_{i=1}^n \mathbb{E}_{\tau_i \sim R} [\tilde{I}_i] .
 \end{aligned}$$

Finally, observe that $\sum_{i=1}^n \tilde{I}_i = \sum_x I_x = I$ since the two sums simply count the same I_x but in different order. This completes the proof of [Lemma 3.2](#). $\square$

LEMMA 3.3. *Let $\mathcal{C}$ be a formula class closed under restrictions, P a monotone proof heuristic with error at most p against $\mathcal{C}$. Then for every $F \in \mathcal{C}$ and every frozen variable x of F it holds that $\mathbb{E}_R[J_x] \leq p$.*

This lemma is in some way the heart of our proof. Its proof studies how the conditional distribution $R(\pi|\alpha)$ differs from the uniform distribution over π and applies two careful coupling arguments. It is also the place where we use that P is monotone. Since its proof

is somewhat non-trivial, let us postpone it to the next section and first finish the proof of the [Main Theorem 1.17](#). [Lemma 3.3](#) has the following consequence:

LEMMA 3.4. $\mathbb{E}_R[pI_x + J_x] \leq p$ for every $x \in V$, and $\mathbb{E}_R[pI + J] \leq pn$.

PROOF. Imagine we run the process SAMPLE-R but pause when x (1) becomes frozen or is (2) is chosen in line 6 as a liquid. Everything that happens before the pause is called *the past*. If (2) happens, then $I_x = 1$, $J_x = 0$ and thus $\mathbb{E}_R[pI_x + J_x | \text{the past}] = \mathbb{E}_R[p \cdot 1 + 0] = p$. Otherwise, if (1) happens, then $I_x = 0$ since x gets frozen before it gets chosen, and $\mathbb{E}_R[pI_x + J_x | \text{the past}] = \mathbb{E}_R[J_x | \text{the past}]$. At the moment of our pause, the sampling process has arrived at a new formula F' , and x is frozen in F' . Since $\mathcal{C}$ is closed under restrictions, $F' \in \mathcal{C}$, too, and we can apply [Lemma 3.3](#) to conclude that $\mathbb{E}_{R_{F'}}[J_x | \text{the past}] = \mathbb{E}_{R_{F'}}[J_x] \leq p$. Thus, $\mathbb{E}_R[pI_x + J_x] \leq p$. $\square$

LEMMA 3.5. $\mathbb{E}_R[I] = \mathbb{E}_Q[I]$.

PROOF. The intuition is that, in both the Q -process and the R -process, the frozen variables do not really matter as their value is already implied by the formula. Thus, whether a process sets frozen x now or later or at the very end influences the position of x but not the rest of the process. Since I counts the number of times a *liquid* variable gets assigned, its expectation is unchanged by the position of frozen variables.

It is a bit tricky to formalize this intuition and requires a bit of terminology. First of all, whether a variable is frozen or liquid does not depend on how F is represented as a CNF formula (or circuit, or whatever), but only on F as a Boolean function. Let $l(F)$ be the set of liquid variables of F and $f(F)$ the set of frozen variables. Note that if $x \in f(F)$ and $F \models (x = b)$ then $l(F|_{x=b}) = l(F)$. That is, assigning a frozen variable can neither freeze nor unfreeze other variables. Define the *liquid reduct* or simply *reduct* of F to be the formula $\text{red}(F)$ obtained by setting all frozen variables to their correct value. Applying $\text{red}(F)$ does not produce “new” frozen variables, by our observation two sentences ago, and therefore, $\text{red}(F)$

has only liquid variables. We say two Boolean functions F and G are *reduct-equivalent* and write $F \equiv_{\text{red}} G$ if $\text{red}(F) = \text{red}(G)$. For example, $(y \vee z)$ and $(x) \wedge (y \vee z)$ are reduct-equivalent, and all formulas with a unique satisfying assignment are reduct-equivalent (if they are defined over the same set of variables); technically, all unsatisfiable formulas over the same set of variables are reduct-equivalent, too, although we never think about unsatisfiable formulas in this context.

We are now ready to prove a slightly more general statement than the lemma. Writing $I(\pi, \alpha, F)$ to highlight the dependency on the ambient formula, we claim:

PROPOSITION 3.6. *It holds that*

$$(3.7) \quad \mathbb{E}_{(\pi, \alpha) \sim Q} [I(\pi, \alpha, F)] = \mathbb{E}_{(\pi, \alpha) \sim R} [I(\pi, \alpha, G)]$$

whenever $F \equiv_{\text{red}} G$.

Applying (3.7) to $F = G$ proves the lemma. $\square$

PROOF (Proof of Proposition 3.6). To prove (3.7), it is best to view Q and R as recursive processes; each samples a pair $(x, b) \in S(F)$ —where R does so uniformly while Q samples $x \in V$ uniformly and then b from the allowed one or two values for x —then outputs (x, b) and recurses on $F|_{x=b}$ (or $G|_{x=b}$, respectively). We will first modify each process by giving them the chance to go idle in a step, i.e., do nothing and recurse on F (or G), and then couple the two modified processes in a way that ensures that both arrive at the same value of I .

We will now describe the coupling. Since $l(F) = l(G)$ we can enumerate them as $x_1, \dots, x_a$; let $y_1, \dots, y_b$ and $z_1, \dots, z_c$ be the frozen variables of F and G , respectively. We write down two sequences of variable-value pairs; the first sequence consists of $2a + 2b$ pairs, the second of $2a + c$ pairs:

$$\begin{aligned} & (x_1, 0), (x_1, 1), \dots, (x_a, 0), (x_a, 1), (y_1, *), (y_1, *), \dots, (y_b, *), (y_b, *) \\ & (x_1, 0), (x_1, 1), \dots, (x_a, 0), (x_a, 1), (z_1, *), (z_2, *), \dots, (z_c, *) \end{aligned}$$

The star simply represents a 0 or 1, to whichever Boolean value the variable in question is frozen to. How does the Q -process select its first pair (x, b) ? Uniformly from the $2a + 2b$ pairs in the first row. How about R ? Uniformly from the $2a + c$ pairs in the second row.

Here is how we couple Q and R . First, the processes sample a common index i uniformly at random in $\{1, \dots, 2a + \max(2b, c)\}$. Second, each process does the following: if “its” row (the row of Q is the first row, the row of R is the second) contains fewer than i pairs, it pauses, i.e., does nothing and recurses on the same formula; if its row contains at least i pairs, let (x, b) be the i^{th} pair. It outputs (x, b) and recurses on $F|_{x=b}$ (for Q) or $G|_{x=b}$ (for R).

Let F' and G' be the respective formulas with which the two processes enter the next recursive call. If $i \leq 2a$ then both processes apply the same assignment $[x = b]$ for a liquid variable x , so $I_x = 1$ in both processes, and $F' \equiv G'$ (it is straightforward to see that if $F \equiv_{\text{red}} G$ then $F|_{x=b} \equiv_{\text{red}} G|_{x=b}$). Otherwise, each process either pauses or has $I_x = 0$; in any case I does not increase, and $F \equiv_{\text{red}} F'$ and $G \equiv_{\text{red}} G'$, so $F' \equiv_{\text{red}} G'$.

To conclude, the two processes are coupled in a way that they do not necessarily work in lockstep, but the two formulas they currently have are reduct-equivalent, and their I increases in lockstep. This concludes the proof of (3.7) and of Proposition 3.6. $\square$

Assuming Lemma 3.3, which we will prove in the next section, we can finish the proof of the Main Theorem 1.17. Above, we have already calculated that the success probability of PPSZ is at least $2^{-\text{the value in (3.1)}}$. The value in (3.1) is $\text{KL}(R||Q) + \mathbb{E}_R[J] \leq p^* \mathbb{E}_R[I] + \mathbb{E}_R[J] = \mathbb{E}_R[pI + J] - (p - p^*) \mathbb{E}_R[I] \leq pn - q \mathbb{E}_Q[I]$. Thus, RANDOMDECODE succeeds with probability at least $2^{-pn + q \mathbb{E}_Q[I]}$. This proves the Main Theorem 1.17.

3.1. Unique to general. We are now ready to prove Lifting Theorem 1.18, which claims that if you can beat PPSZ for UNIQUE- $\mathcal{C}$ -SAT, then you can beat it for $\mathcal{C}$ -SAT.

PROOF (Proof of Lifting Theorem 1.18). We will prove Point (i) of the theorem. The other two points are similar. We begin by reminding the reader of the Main Theorem 1.17: suppose proof

heuristic P has error rate at most p against a class $\mathcal{C}$; we assume that $p > p^* := \frac{2^{-\log(e)}}{2}$ and write $q := p - p^*$. If $F \in \mathcal{C}$ is satisfiable then

$$(3.8) \Pr[\text{RANDOMDECODE}(F, P) \text{ successful}] \geq 2^{-pn+q\mathbb{E}_Q[I]} ,$$

We will now follow a meet-in-the-middle argument depending on whether $\mathbb{E}_Q[I]$ is large or small. Intuitively, if it is large, then (3.8) is better than 2^{-pn} . If it is small, then F is close to having a unique satisfying assignment and we can apply our improved bound for Unique-SAT. To make this formal, let $\delta > 0$ be a fixed number, to be determined later. We distinguish two cases. In the first case, if $\mathbb{E}_Q[I] \geq \delta \cdot n$, we bound (3.8) by

$$(3.9) \Pr[\text{RANDOMDECODE}(F, P) \text{ successful}] \geq 2^{-pn+q\delta n} .$$

In the second case, we assume that $\mathbb{E}_Q[I] \leq \delta n$. In particular, $I(\pi, \alpha) \leq \delta n$ for *some* permutation π and satisfying assignment α . This means that there is a partial assignment ρ fixing δn variables such that $F|_\rho$ has a unique satisfying assignment. When running RANDOMDECODE on F , with probability $\binom{n}{\delta n}^{-1} \cdot 2^{\delta n}$ the first δn steps produce exactly ρ , and the remaining $(1 - \delta)n$ steps are like running $\text{RANDOMDECODE}(F|_\rho, P)$. $F|_\rho$ has a unique solution α' , and thus, $\text{RANDOMDECODE}(F|_\rho, P)$ finds α' with probability at least $2^{-(p+\epsilon)(1-\delta)n}$. Altogether,

$$(3.10) \quad \begin{aligned} \Pr[\text{RANDOMDECODE} = \alpha] &\geq \binom{n}{\delta n}^{-1} \cdot 2^{-\delta n} \cdot 2^{(-p+\epsilon)(1-\delta)n} \\ &\geq 2^{-(H(\delta)+\delta+p-\epsilon)n} \end{aligned}$$

where H is the binary entropy function. The bound in (3.9) increases with δ while (3.10) decreases. A quick calculation shows that by choosing $\delta := \frac{\epsilon}{c \log(1/\epsilon)}$ for an appropriate constant c ($c = 4$ should work) we can achieve that (3.10) is at least $2^{-(p+c'\epsilon)n}$ for some $c' > 0$. and Thus, both (3.9) and (3.10) are at least $2^{-(p+c'\epsilon)n}$ for $\epsilon' = C \cdot \frac{\epsilon}{\log(1/\epsilon)}$ for some $C > 0$. The exact value of C depends on $q = p - p^*$ and thus on how far away the error probability p is from the threshold p^* . The proofs of the other two points of the theorem are similar. $\square$

4. Proof of Lemma 3.3

LEMMA (Lemma 3.3, restated.). *Let $\mathcal{C}$ be a formula class closed under restrictions, P a monotone proof heuristic with error at most p against $\mathcal{C}$. Then for every $F \in \mathcal{C}$ and every frozen variable x of F it holds that $\mathbb{E}_R[J_x] \leq p$.*

PROOF. In the expression $\mathbb{E}_R[J_x]$, both π and α are random. We obtain a fixed α by conditioning:

$$\mathbb{E}_{(\pi, \alpha) \sim R} [J_x(\pi, \alpha)] = \mathbb{E}_{\alpha \sim R} [\mathbb{E}_{\pi \sim R | \alpha} [J_x(\pi, \alpha)]] .$$

We will prove a stronger statement. Indeed,

$$(4.1) \quad \mathbb{E}_{\pi \sim R | \alpha} [J_x(\pi, \alpha)] \leq p \quad \text{for every } \alpha \in \text{sat}(F)$$

Before proving (4.1) we have to introduce some terminology. For a formula F over variable set V , recall that $S(F, V)$ is the set of all pairs $(x, b) \in V \times \{0, 1\}$ for which $F|_{x=b}$ is satisfiable. Note that if F is satisfiable then $|S(F, V)|$ is n plus the number of liquid variables. We call a function $g : 2^V \rightarrow \mathbb{R}$ *monotone* if $g(A) \leq g(B)$ for any $A \subseteq B \subseteq V$. Let $x \in V$ be a fixed variable, $\pi \in \text{Sym}(V)$ a permutation. We denote by $W(\pi)$ the set of variables appearing after x in π . Observe that $J_x(\pi, \alpha)$ only depends on $W(\pi)$, not on the particular order of the variables coming before x and of those coming after x .

OBSERVATION 4.2. *If P is a monotone heuristic and x is a frozen variable, then J_x is a monotone function in W .*

PROOF. Let $x \in V$ and $\alpha \in \text{sat}(F)$. Let π and π' be two permutations with $W(\pi) \subseteq W(\pi')$. We have to show that $J_x(\pi, \alpha) \leq J_x(\pi', \alpha)$. For this we consider $B := V \setminus \{x\} \setminus W(\pi)$ and $B' := V \setminus \{x\} \setminus W(\pi')$, the sets of variables appearing *before* x in the permutations π and π' , respectively. It holds that $B' \subseteq B$. We write F_B to denote the formula that we obtain by setting each $y \in B$ according to its value under α ; similarly $F_{B'}$. Now if $J_x(\pi', \alpha) = 1$, the claimed inequality holds. Otherwise, $J_x(\pi', \alpha) = 0$, and since x is frozen this means that $P(F_{B'}) = \alpha(x)$, i.e., P can infer the

correct value of x when given $F_{B'}$. Since P is a monotone proof heuristic and F_B is obtained from $F_{B'}$ by further setting the variables in $B \setminus B'$, it follows that $P(F_{B'}) = \alpha(x)$, too, so $J_x(\pi, \alpha) = 0$. $\square$

For two strings σ, π , we write $\sigma \preceq \pi$ if σ is a prefix of π . A permutation π on set V of size n can be viewed as a string in V^n without repeated letters. A string $\sigma \in V^*$ without repeated letters is called a *partial permutation*. If D is a distribution over permutations on V and σ is a partial permutation, we write $D(\sigma) := \sum_{\pi: \sigma \preceq \pi} D(\pi) = \Pr_{\pi \sim D}[\sigma \preceq \pi]$.

DEFINITION 4.3. *Let D be a distribution over permutations on V , and let $x \in V$. We say D delays x if for all $y \in V$ and all partial permutations σ not containing x or y , it holds that $D(\sigma x) \leq D(\sigma y)$.*

Informally, at every stage, x is among the least likely elements to come next. For example, the uniform distribution delays x ; so does any distribution that samples a permutation of $V \setminus \{x\}$ and places x at the end. [Lemma 3.3](#) will follow from the next two lemmas:

LEMMA 4.4. *If x is a frozen variable of F then the distribution $(R|\alpha)$ delays x .*

Here, $(R|\alpha)$ is the distribution on permutations output by the R -process conditioned on R outputting the satisfying assignment α , i.e., $(R|\alpha)(\pi) = R(\pi, \alpha)/R(\alpha)$.

LEMMA 4.5. *Let V be a finite set, $x \in V$, D a distribution over permutations of V that delays x , and $f : V \rightarrow \mathbb{R}$ a monotone function. Denote by $W = W(\pi)$ the set of elements coming after x in π . Then*

$$\mathbb{E}_{\pi \sim D}[f(W)] \leq \mathbb{E}_{\pi \sim \mathcal{U}}[f(W)] ,$$

where $\mathcal{U}$ is the uniform distribution over permutations.

With [Lemma 4.4](#) and [Lemma 4.5](#), we can quickly finish the proof of [Lemma 3.3](#), or, to be more precise, the proof of [\(4.1\)](#):

$$((4.1), \text{restated}) \quad \mathbb{E}_{\pi \sim R|\alpha}[J_x(\pi, \alpha)] \leq p .$$

By Lemma 4.4, $(R|\alpha)$ delays x . By Observation 4.2, J is a monotone function in W , where $W = W(\pi)$ is the set of elements coming after x in π . Thus, Lemma 4.5 implies $\mathbb{E}_{\pi \sim R|\alpha}[J_x(\pi, \alpha)] \leq \mathbb{E}_{\pi \sim \mathcal{U}}[J_x(\pi, \alpha)] \leq p$, where the latter inequality follows from the error rate of P . $\square$

PROOF (of Lemma 4.4). By the assumption in the lemma, x is frozen; by Definition 4.3, we have to show that $(R|\alpha)(\sigma x) \leq (R|\alpha)(\sigma y)$ where σ is a partial permutation not containing x nor y . This is equivalent to $\frac{R(\sigma x, \alpha)}{R(\alpha)} \leq \frac{R(\sigma y, \alpha)}{R(\alpha)}$ and thus equivalent to showing $R(\sigma x, \alpha) \leq R(\sigma y, \alpha)$. We have not formally introduced the notation $R(\sigma, \alpha)$ for a partial permutation σ but the meaning should be clear: it is the probability that the R -process outputs the assignment α and some permutation π with $\sigma \preceq \pi$. Formally, $R(\sigma, \alpha) = \sum_{\pi: \sigma \preceq \pi} R(\pi, \alpha)$.

Assume first that σ is empty, so we have to show that $R(x, \alpha) \leq R(y, \alpha)$. Consider the following alternative but equivalent way to sample R : order the s elements of $S(F, V)$ randomly into a sequence $\tau = (x_1, b_1), \dots, (x_s, b_s)$ and then add the unit clauses $(x_i = b_i)$ to F , in this order, skipping a unit clause if adding it would make F unsatisfiable. This adds n non-contradictory unit clauses in some order $(x_{i_1} = b_{i_1}), \dots, (x_{i_n} = b_{i_n})$ and thus defines a permutation π of V and an assignment α . The pair (π, α) has distribution R . There are $|S(F, V)|!$ such sequences.

Let $T_{z, \alpha}$ denote the set of all such sequences τ that (1) result in α and (2) place z at the beginning of π . So $R(z, \alpha) = \frac{|T_{z, \alpha}|}{|S(F, V)|!}$. Since the first unit clause $(x_1 = b_1)$ in a sequence is always consistent with F and thus the alternative R -process just described surely accepts the first pair, every sequence in $T_{z, \alpha}$ starts with $(z = \alpha(z))$.

We have to show that $R(x, \alpha) \leq R(y, \alpha)$, which is equivalent to $|T_{x, \alpha}| \leq |T_{y, \alpha}|$. For a sequence $\tau \in T_{x, \alpha}$ define $f(\tau)$ to be the sequence τ' where we switch the positions of $(x = \alpha(x))$ and $(y = \alpha(y))$ (note that both must appear in τ , and $(x = \alpha(x))$ appears at the beginning). A minute of thought shows that the sequence $f(\tau)$ leads to α as well (the key observation is that x is frozen, so logically $(x = \alpha(x))$ is already present in F , whether it occurs at the beginning of τ or not). Thus, $f(\tau) \in T_{y, \alpha}$ and we have just

defined an injection from $T_{x,\alpha}$ into $T_{y,\alpha}$. This shows that $|T_{x,\alpha}| \leq |T_{y,\alpha}|$ and thus $R(x, \alpha) \leq R(y, \alpha)$.

If σ is not empty, we write $\alpha = \alpha_\sigma \alpha_{\bar{\sigma}}$, where α_σ is the α restricted to the variables appearing in σ , and $\alpha_{\bar{\sigma}}$ is the rest. Write $F' := F|_{\alpha_\sigma}$. Now $R(\sigma z, \alpha)$ is the probability that SAMPLE-R follows σ and α in its first $|\sigma|$ steps, times $R_{F'}(z, \alpha_{\bar{\sigma}})$. Thus, we have reduced the non-empty σ case to the empty σ case. $\square$

LEMMA (Lemma 4.5, restated.). *Let V be a finite set, $x \in V$, D be a distribution over permutations of V that delays x , and $f : V \rightarrow \mathbb{R}$ be a monotone function. Denote by $W = W(\pi)$ the set of elements coming after x in π . Then*

$$\mathbb{E}_{\pi \sim D} [f(W)] \leq \mathbb{E}_{\pi \sim \mathcal{U}} [f(W)],$$

where $\mathcal{U}$ is the uniform distribution over permutations.

PROOF. Let W_D denote a random variable distributed like $W(\pi)$ with $\pi \sim D$, and similarly $W_{\mathcal{U}} = W(\pi)$ where π is uniform. Below, we define a process SAMPLE-W which simultaneously samples W_D and $W_{\mathcal{U}}$ and guarantees $W_D \subseteq W_{\mathcal{U}}$. In other words, SAMPLE-W defines a coupling under which $W_D \subseteq W_{\mathcal{U}}$ always holds. We write $D(z|\sigma) := D(\sigma z|\sigma) = \frac{D(\sigma z)}{D(\sigma)}$. This is the probability that z is chosen next, conditioned on σ having been sampled so far.

The process SAMPLE-W clearly samples W_D from the correct distribution. Note that an element z gets removed from $W_{\mathcal{U}}$ whenever $t < D(x|\sigma)$, and then a uniformly random element is removed. Also, the process terminates when x has been removed from W_D . Obviously, it will be removed from $W_{\mathcal{U}}$ in the same iteration. So W_D and $W_{\mathcal{U}}$ have the correct distribution. Lastly, since $D(x|\sigma) \leq D(z|\sigma)$, when the element z is removed from $W_{\mathcal{U}}$, it has already been removed from W_D . Thus, $W_D \subseteq W_{\mathcal{U}}$ holds in every step. Thus, $f(W_D) \leq f(W_{\mathcal{U}})$ with probability 1 and therefore $\mathbb{E}_{\pi \sim D} [f(W)] \leq \mathbb{E}_{\pi \sim \mathcal{U}} [f(W)]$. $\square$

5. Constraint satisfaction problems

Informally speaking, constraint satisfaction problems (CSPs) are just like SAT, but variables are able to take on more than two

Algorithm 5 Sampling W_D and W_U

```

1: procedure SAMPLE-W( $V, x$ )
2:    $\sigma :=$  the empty string
3:    $W_D = W_U = V$ 
4:   while  $x \in W_D$  do
5:      $(z, t) \in V \times [0, 1]$ , uniformly at random
6:     if  $t < D(z|\sigma)$  and  $z \in W_D$  then
7:       remove  $z$  from  $W_D$ 
8:       append  $z$  to  $\sigma$ 
9:     end if
10:    if  $t < D(x|\sigma)$  then
11:      remove  $x$  from  $W_U$ 
12:    end if
13:  end while
14:  return  $W_D, W_U$ 
15: end procedure

```

values. Formally, a (d, k) -CSP F is defined by a set of variables $V = \{x_1, \dots, x_n\}$, each of which can take on values in some *domain* $[d] = \{1, \dots, d\}$, and a conjunction of constraints $C_1 \wedge \dots \wedge C_m$, where each constraint C is of the form

$$(5.1) \quad (y_1 \neq c_1 \vee y_2 \neq c_2 \vee \dots \vee y_{k'} \neq c_{k'})$$

where $y_i \in V$, $c_i \in [d]$, and $k' \leq k$. Thus, each constraint involves $k' \leq k$ variables and forbids one of the $d^{k'}$ possible assignments to them. Each $y_i \neq c_i$ in (5.1) is called a *literal*. An assignment $\alpha : V \rightarrow [d]$ satisfies the literal $x \neq b$ if $\alpha(x) \neq b$; it satisfies (5.1) if it satisfies at least one literal therein; it satisfies F if it satisfies all $C_1, \dots, C_m$. Assuming some implicit ordering on the n variables in V , we identify $\alpha : V \rightarrow [d]$ with $\alpha \in [d]^n$. We denote by $\text{sat}(F) \subseteq [d]^n$ the set of satisfying assignments. Of course, one might be tempted to define a more general version where each constraint C is of the form $(y_1, \dots, y_k) \in S$ for some $S \subseteq [d]^k$, i.e., each constraint specifies a set of allowed values for the k variables involved; however, it is easy to see that $(y_1, \dots, y_k) \in S$ can be written as a conjunction of $d^k - |S|$ many constraints like (5.1). Since we assume that d and k are constants, this leads to

only a constant factor overhead in encoding more sophisticated constraints. Note that $(2, k)$ -CSP is the same as SAT (just that the domain is $\{1, 2\}$ and not $\{0, 1\}$).

Translating a result (like an algorithm) from SAT to CSP is sometimes straightforward; for example, Schönning’s random walk algorithm (Schönning 1999) generalizes flawlessly from k -SAT to (d, k) -CSP, with no surprises. Sometimes, it takes significant extra work, like analyzing PPSZ for (d, k) -CSP, both in the Unique-SAT and General-SAT setting, which was done by Hertli, Hurbain, Millius, Moser, Scheder & Szedlák (2016). Their work on (d, k) -CSPs with multiple solutions is a generalization of the result of Hertli (2011) to (d, k) -CSPs. Analogously, this section of our work can be seen as a simplification of Hertli *et al.* (2016) or as a generalization of our proof of Main Theorem 1.17 to (d, k) -CSPs.

5.1. Proof heuristics for (d, k) -CSPs. Let $F = C_1 \wedge \dots \wedge C_m$ be a (d, k) -CSP over variable set V , let $x \in V$ and $b \in [d]$. Similar to the Boolean case, we write $F \models (x \neq b)$ if all $\alpha \in \text{sat}(F)$ have $\alpha(x) \neq b$. By $F|_{x=b}$ we denote the CSP resulting from F by replacing x by the constant b ; that means, when a constraint C contains the literal $x \neq b$ then this literal becomes false and we remove it from C ; if C contains $x \neq b'$ for some $b' \neq b$, then the literal $x \neq b$ becomes true and we remove C from F . We define the set of *satisfying colors* of x as

$$(5.2) \quad S_x(F) := \{c \in [d] \mid \exists \alpha \in \text{sat}(F) \alpha(x) = c\} .$$

Equivalently, it is the set of $c \in [d]$ for which $F|_{x=c}$ is satisfiable.

How should we define proof heuristics in the non-Boolean case? It should never tell us something incorrect but might sometimes answer “I don’t know”. Formally, P is a deterministic algorithm that takes as input a CSP F and a variable x and returns a set $P_x(F) \subseteq [d]$ of *plausible* values such that

$$(5.3) \quad S_x(F) \subseteq P_x(F) .$$

It rules out values “conservatively”—if $c \notin P_x(F)$ we can be sure that $F|_{x=c}$ is unsatisfiable, but if $c \in P_x(F)$ then $F|_{x=c}$ might or might not be satisfiable. Note that our heuristic P in the Boolean

case, which answers $P(F, x) \in \{0, 1, ?\}$, fits into this framework by replacing the answers with $\{0\}$, $\{1\}$, and $\{0, 1\}$, respectively. The proof heuristic P is *monotone* if

$$(5.4) \quad P_x(F|_{y=c}) \subseteq P_x(F)$$

for all formulas F and all variables $x \neq y$ and colors $c \in [d]$.

EXAMPLE 5.5. The heuristic $P^{(w)}$. For formula F , variable x , and color c , we write $F \models_w (x \neq c)$ if there is a subset G of at most w constraints of F with $G \models (x \neq c)$. Then

$$P_x^{(w)} := \{c \in [d] \mid F \not\models_w (x \neq c)\}.$$

This is the heuristic used by Hertli *et al.* (2016). It is a monotone heuristic. $\diamond$

We are now ready to state PPSZ formally for (d, k) -CSPs. Note that we refrain from stating it as an encoding / decoding scheme because the non-Boolean variables do not fit well into this framework. See Algorithms 6 and 7.

Algorithm 6 Generic PPSZ Procedure for Fixed Permutation

```

1: procedure ppsz_fixed( $F, P, \pi$ )
2:    $\beta :=$  the empty assignment on  $V$ 
3:   for  $x \in V$  in the order of  $\pi$  do
4:     Select  $c \in P_x(F|_\beta)$  uniformly at random
5:      $\beta(x) := c$ 
6:   end for
7:   return  $\beta$ 
8: end procedure

```

Algorithm 7 Generic PPSZ Procedure

```

1: procedure ppsz( $F, P$ )
2:    $\pi :=$  a uniformly random permutation of  $\text{var}(F)$ 
3:   return ppsz_fixed( $F, P, \pi$ )
4: end procedure

```

It is understood that `ppsz_fixed` returns `failure` if the set $P(F|_{\beta}, x)$ is empty at any point in Line 4, or if β does not satisfy F once line 7 is reached. Let π a permutation of V , $x \in V$, and $\alpha \in \text{sat}(F)$. We write $F|_{\pi, x, \alpha}$ to denote the formula that arises if we fix y to $\alpha(y)$ for all variables y that appear before x in π . Write

$$(5.6) \quad S_x(F, \pi, \alpha) \quad := S_x(F|_{\pi, x, \alpha})$$

$$(5.7) \quad P_x(F, \pi, \alpha) \quad := P_x(F|_{\pi, x, \alpha}) .$$

If F is understood, we might write $S_x(\pi, \alpha)$ and $P_x(\pi, \alpha)$ for brevity. We can give an exact formula for the success probability of PPSZ in terms of these expressions:

OBSERVATION 5.8. *Let α be a satisfying assignment of F . Then*

$$\Pr[\text{ppsz_fixed}(F, P, \pi) = \alpha] = \prod_{x \in \text{var}(F)} \frac{1}{|P_x(F, \pi, \alpha)|} .$$

Using this observation, one calculates:

$$\begin{aligned} \Pr[\text{ppsz}(F, P) = \alpha] &= \mathbb{E}_{\pi} \left[\prod_{x \in \text{var}(F)} \frac{1}{|P_x(F, \pi, \alpha)|} \right] \\ &= \mathbb{E}_{\pi} \left[2^{-\sum_{x \in \text{var}(F)} \log |P_x(F, \pi, \alpha)|} \right] \\ (5.9) \quad &\geq 2^{-\sum_{x \in \text{var}(F)} \mathbb{E}_{\pi} [\log |P_x(F, \pi, \alpha)|]} . \end{aligned}$$

Now if α is the unique satisfying assignment, then we are left with bounding $\mathbb{E}_{\pi} [\log |P_x(F, \pi, \alpha)|]$ from above, which is exactly what the first part of [Hertli et al. \(2016\)](#) deals with. In this work, we strive for a more abstract result, asking when and how the analysis for the Unique-CSP can be extended to General-CSP. For this, we again need a way to gauge the power of P .

DEFINITION 5.10. *Let P be a proof heuristic. We say P has error rate at most p against (d, k) -CSP formulas if $\mathbb{E}_{\pi} [\log |P_x(F, \pi, \alpha)|] \leq p$ for all (d, k) -CSP formulas F , all $\alpha \in \text{sat}(F)$, and all $x \in \text{var}(F)$ for which $|S_x(F)| = 1$.*

This is just as in the Boolean case. The condition $|S_x(F)| = 1$ means that x is frozen, i.e., has only one satisfying value, and $\mathbb{E}_\pi[\log |P_x(F, \pi, \alpha)|]$ measures how many “bits” we will need, on expectation, to encode the value of x in α . One difficulty in generalizing [Main Theorem 1.17](#) to (d, k) -CSP is that there are not only liquid and frozen variables—if F is satisfiable then a variable can have 1, 2, $\dots$, or d possible values. More concretely, for a particular proof heuristic it might well be possible to prove an upper bound on $\mathbb{E}_\pi[\log |P_x(F, \pi, \alpha)|]$ for variables x with $|S_x(F)| = 2$. Alas, we do not know how to use such a bound and thus will treat all variables with $|S_x(F)| > 1$ as “liquid”.

OBSERVATION 5.11. *If F is a (d, k) -CSP formula with a unique satisfying assignment α and P has error rate at most p against (d, k) -CSP formulas, then $\Pr[\text{ppsz}(F, P) = \alpha] \geq 2^{-pn}$.*

THEOREM 5.12 (Theorem 1.2 from [Hertli et al. 2016](#)). *Let F be a satisfiable (d, k) -CSP and P a monotone proof heuristic of error rate at most p against (d, k) -CSP formulas for $p \geq p^* := \log(d) - \frac{\log(e)}{2}$. Then $\text{ppsz}(F, P)$ finds a satisfying assignment of F with probability at least 2^{-pn} .*

[Hertli et al. \(2016\)](#) prove this theorem by a machinery that is similar to [Hertli \(2011\)](#) but arguably even more technical. In this section we give a simpler proof. As an aside, note that in the Boolean case $d = 2$, all known upper bounds on error rates against k -CNF formulas are well above the threshold p^* and therefore we can claim that PPSZ works on General- k -SAT just as well as on Unique- k -SAT. The same cannot be said for general d —the error rates proved by [Hertli et al. \(2016\)](#) for $P^{(w)}$ against (d, k) -CSPs are *smaller* than p^* when $k = 2$ and $d \geq 3$ or $k = 3$ and $d \geq 6$. Thus, for certain values of (d, k) our current best bounds for General- (d, k) -CSP are significantly worse than for Unique- (d, k) -CSP. However, when $p \geq p^*$ then [Theorem 5.12](#) holds and we also get a lifting result similar to [Lifting Theorem 1.18](#) in the Boolean case.

5.2. A failed attempt at Theorem 5.12. The first thing to try is of course to naively generalize our technique from the Boolean case to the (d, k) -CSP case. We can easily define a Q -process and an R -process. For a formula, let

$$(5.13) \quad S(F) = \{(x, c) \in V \times [d] \mid c \in S_x(F)\} ,$$

so $|S(F)| = \sum_x |S_x(F)|$. The Q -process iteratively (1) selects $x \in V(F)$ uniformly at random and then $c \in S_x(F)$, (2) outputs (x, c) , and then (3) continues on $F|_{x=c}$; the R -process differs from Q by selecting $(x, c) \in S(F)$ uniformly at random.

OBSERVATION 5.14. *The distributions Q and R are given by*

$$(5.15) \quad Q(\pi, \alpha) = \frac{1}{n!} \prod_x \frac{1}{|S_x(F, \pi, \alpha)|}$$

$$(5.16) \quad R(\pi, \alpha) = \prod_x \frac{1}{|S(F|_{x, \pi, \alpha})|} .$$

Mimicking our proof of the Boolean case, we write

$$\begin{aligned} \Pr[\text{success}] &= \sum_{\alpha \in \text{sat}(F)} \mathbb{E}_\pi \left[\prod_x \frac{1}{|P_x(\pi, \alpha)|} \right] \\ &= \sum_{\pi, \alpha} \frac{1}{n!} \prod_x \frac{1}{|S_x(\pi, \alpha)|} \frac{|S_x(\pi, \alpha)|}{|P_x(\pi, \alpha)|} \\ &= \sum_{\pi, \alpha} Q(\pi, \alpha) \frac{|S_x(\pi, \alpha)|}{|P_x(\pi, \alpha)|} \\ &= \mathbb{E}_R \left[\frac{Q(\pi, \alpha)}{R(\pi, \alpha)} \prod_x \frac{|S_x(\pi, \alpha)|}{|P_x(\pi, \alpha)|} \right] \\ &= \mathbb{E}_R \left[2^{-\log\left(\frac{R(\pi, \alpha)}{Q(\pi, \alpha)}\right) - \sum_x \log_2\left(\frac{|P_x(\pi, \alpha)|}{|S_x(\pi, \alpha)|}\right)} \right] \\ &\geq 2^{-\text{KL}(R||Q) - \sum_x \mathbb{E}_R[\log_2\left(\frac{|P_x(\pi, \alpha)|}{|S_x(\pi, \alpha)|}\right)]} , \end{aligned}$$

and then strive to bound

$$(5.17) \quad \text{KL}(R||Q) + \sum_x \mathbb{E}_R \left[\log_2 \left(\frac{|P_x(\pi, \alpha)|}{|S_x(\pi, \alpha)|} \right) \right] .$$

But the trouble already starts with the second sum. As in the Boolean case, we let PPSZ run until either (1) x freezes or (2) x gets processed. If (1) happens then $|S_x(\alpha)| = 1$ and we try to bound $\mathbb{E}_R[|P_x|]$. If (2) happens then we do not have any handy upper bound on $\mathbb{E}_R[|P_x|]$. In the Boolean case this was not a big problem— $|S_x| = 2$ in that case and $|P_x| = 2$ and the logarithm would vanish. But in the d -ary case it might be that $|S_x| = 2$, while $|P_x| = d$; we don't know!

It turns out that writing $C_x(\pi, \alpha)$ as the sum of $I_x(\pi, \alpha)$ and $J_x(\pi, \alpha)$, which worked so well in the Boolean case, will not work for here—its analog would be writing $\frac{1}{|P_x(\pi, \alpha)|}$ as the product of $\frac{1}{|S_x(\pi, \alpha)|}$ and $\frac{|S_x(\pi, \alpha)|}{|P_x(\pi, \alpha)|}$, but this will bring us on the wrong track; we have to open up (5.17) and re-work it by hand.

5.3. Proof of Theorem 5.12. We rewrite the success probability of PPSZ as an expectation with respect to our particular distribution R defined by the R -process in the previous section. In contrast to the Boolean case, we do not use the distribution Q at all.

$$\begin{aligned} \Pr[\text{success}] &= \sum_{\alpha} \mathbb{E}_{\pi} \left[\prod_x \frac{1}{|P_x(\pi, \alpha)|} \right] = \sum_{\pi, \alpha} \frac{1}{n!} \prod_x \frac{1}{|P_x|} \\ &= \mathbb{E}_{(\pi, \alpha) \sim R} \left[\frac{1}{n! R(\pi, \alpha)} \prod_x \frac{1}{|P_x|} \right] \\ &\geq 2^{-\mathbb{E}_R[\log(n! R(\pi, \alpha))] - \sum_x \mathbb{E}_R[\log |P_x|]} . \end{aligned}$$

After this application of Jensen's inequality, our task is to bound

$$(5.18) \quad \mathbb{E}_R[\log(n! R(\pi, \alpha))] + \sum_x \mathbb{E}_R[\log |P_x|]$$

from above. Recall that $S_x = S_x(\pi, \alpha)$ is the set of satisfying colors for x at the point in time when x is processed. Let L_x be the indicator variable that is 1 if $|S_x| \geq 2$ and 0 otherwise. We call x *frozen in F* if $|S_x(F)| = 1$, and *liquid* otherwise. Thus, L_x is the indicator variable of x being liquid when it is selected in R . Let $\tilde{L}_i$ be the indicator variable that is 1 if the variable selected by R in its i^{th} step is liquid. Note that $\tilde{L}_i$ and L_x are two different ways to count the same things.

PROPOSITION 5.19. $\mathbb{E}_R[\log(n!R(\pi, \alpha))] < \frac{-\log(e)}{2} \cdot \sum_x \mathbb{E}_R[L_x]$.

PROOF. Consider the i^{th} step of the R -process. Let F_i be the formula at the beginning of step i , let $n_i := n - i + 1$ be the number variables in F_i , let $s_i := |S(F_i)| = \sum_{x \in \text{var}(F_i)} |S_x(F_i)|$. All of these are random variables with respect to the underlying probability distribution R (except the n_i , of course, which are known beforehand). By (5.16) we have $\log(n!R(\pi, \alpha)) = \sum_{i=1}^n \log\left(\frac{n_i}{s_i}\right)$. Let us imagine the R -process is at the beginning of the i^{th} step and let us condition everything on what happened in the $i - 1$ first steps. The random variable s_i is now a constant. Let us write s and n instead of s_i and n_i , to simplify notation. Let f be the number of frozen variables in F_i (now also a constant). We see that $\mathbb{E}[L_i] = 1 - \frac{f}{s}$. Also, every liquid variable has at least 2 possible colors,² thus $s \geq f + 2(n - f) = 2n - f$ and $f \geq 2n - s$. Therefore,

$$\mathbb{E}[L_i] = 1 - \frac{f}{s} \leq 1 - \frac{2n - s}{s} = \frac{2s - 2n}{s}.$$

or equivalently, $\frac{s-n}{s} \geq \frac{\mathbb{E}[L_i]}{2}$. Next, we will bound $\log\left(\frac{n}{s}\right)$ from below:

$$\begin{aligned} \log\left(\frac{n}{s}\right) &= \log\left(1 - \frac{s-n}{s}\right) \\ &< -\log(e) \cdot \frac{s-n}{s} \\ &\leq \frac{-\log(e)}{2} \mathbb{E}[L_i]. \end{aligned}$$

Putting everything together, we get

$$\begin{aligned} \mathbb{E}_R[\log(n!R(\pi, \alpha))] &= \sum_{i=1}^n \mathbb{E}\left[\log\left(\frac{n_i}{s_i}\right)\right] \\ &< \frac{-\log(e)}{2} \sum_i \mathbb{E}_R[L_i] \\ &= \frac{-\log(e)}{2} \sum_x \mathbb{E}_R[L_x], \end{aligned}$$

²Here we use another trivial (lower) bound; we feel that simultaneously using the estimates $|P_x| \leq d$ and $|S_x| \geq 2$ for a liquid variable is too pessimistic and thus suboptimal. However, we do not see a way around this at the moment.

which proves [Proposition 5.19](#). $\square$

This bounds the first term of [\(5.18\)](#) from above. For the second term, we need following lemma, which we prove below.

LEMMA 5.20. $\mathbb{E}_R[\log |P_x|] \leq p + \mathbb{E}_R[(\log(d) - p)L_x]$.

Together, [Proposition 5.19](#) and [Lemma 5.20](#) imply

$$\begin{aligned} (5.18) &\leq pn + \sum_x \mathbb{E}_R \left[\left(\log(d) - p - \frac{\log(e)}{2} \right) L_x \right] \\ &\leq pn - q \sum_x \mathbb{E}_R[L_x] . \end{aligned}$$

This proves [Theorem 5.12](#) but in fact proves a stronger statement.

THEOREM 5.21 (Unique-to-General Lifting for CSP). *Suppose $p > p^*$ and let $q := p - p^*$. Then $\text{ppsz}(F, P)$ finds a satisfying assignment of F with probability at least $2^{-pn - q \sum_x \mathbb{E}_R[L_x]}$.*

Completely analogous to [Lifting Theorem 1.18](#), we obtain the following corollary:

COROLLARY 5.22. *If there is an $\epsilon > 0$ and a (Monte Carlo) algorithm solving Unique- (d, k) -CSP in time $2^{pn - \epsilon n}$, then there is a Monte Carlo algorithm solving general- (d, k) -CSP in time $2^{pn - \epsilon' n}$, for some $\epsilon' > 0$.*

Recently, [Li & Scheder \(2021\)](#) gave a modified version of PPSZ for Unique- (d, k) -CSP that achieves an exponentially (but rather small) improvement for all k and $d \geq 3$; [Zamir \(2021\)](#) gave an algorithm that is substantially faster than PPSZ on Unique- $(d, 2)$ -CSP for $k \geq 5$. The above corollary means that both results translate to improving the corresponding *general* CSP, albeit with a smaller improvement.

PROOF (of [Lemma 5.20](#)). Let the R -process run until one of two things happens: (1) x freezes, i.e., $|S_x(F)|$ drops to 1; (2) x is selected as the next variable in π . If (2) happens first, then $L_x = 1$

and the right-hand side becomes $\log(d)$. Since $|P_x| \leq d$ always,³ the inequality holds. If (1) happens first, then $L_x = 0$ and the right-hand side evaluates to p . Let F' be the restricted formula after (1) has happened. The left-hand side becomes $\mathbb{E}_R[\log P_x(F', \pi, \alpha)]$. Note that x is frozen in F' .

LEMMA 5.23. *If P is a monotone proof heuristic against of error rate at most p against (d, k) -CSP formulas, F is a (d, k) -CSP formula, and x is frozen in F , then*

$$(5.24) \quad \mathbb{E}_R[\log |P_x(F, \pi, \alpha)|] \leq p .$$

Using this lemma, we see that in case (1) the left-hand in [Lemma 5.20](#) is at most p , and the right-hand side is exactly p . This concludes the proof of [Lemma 5.20](#). $\square$

It remains to prove [Lemma 5.23](#).

PROOF (of [Lemma 5.23](#)). We write $P_x = P_x(F, \pi, \alpha)$ and

$$(5.25) \quad \mathbb{E}_{(\pi, \alpha) \sim R}[\log |P_x|] = \mathbb{E}_{\alpha \sim R} \left[\mathbb{E}_{\pi \sim (R|\alpha)}[\log |P_x|] \right] ,$$

where the conditional distribution is $(R|\alpha)(\pi) = R(\pi, \alpha)/R(\alpha)$ and $R(\alpha) = \sum_{\pi} R(\pi, \alpha)$. We have to show that [\(5.25\)](#) $\leq p$. In fact, we will show that

$$(5.26) \quad \mathbb{E}_{\pi \sim R|\alpha}[\log |P_x|] \leq p , \text{ for every fixed } \alpha \in \text{sat}(F) .$$

Using the terminology developed in [Section 4](#), the inequality will follow from proving that $R|\alpha$ delays x (in the meaning of [Definition 4.3](#)), the fact that $|P_x(\pi, \alpha)|$ is monotone in the variables coming after x in π , and [Lemma 4.5](#).

LEMMA 5.27. *Let F be a (d, k) -CSP formula and x be a frozen variable in F . Then the distribution $R|\alpha$ delays x .*

The proof is almost identical to that of [Lemma 4.4](#). One simply checks that nothing in the proof of [Lemma 4.4](#) uses the fact that there are only two truth values. This concludes the proof of [Lemma 5.23](#). $\square$

³Here we use the trivial upper bound $|P_x| \leq d$, which might be one point where our proof is still suboptimal.

6. Conclusion and open questions

We would like to find a proof that works for all monotone proof heuristics, for arbitrarily small error rates p . More modestly, it would be nice if we could extend the proof of this paper to cover all values d, k for the concrete proof heuristic $P^{(w)}$. There are two points in the proof where we bound things too generously: first, in the proof of [Lemma 5.20](#) we bound $\log |P_x| \leq \log(d)$, which is of course true. However, it is too pessimistic on expectation. For example, suppose $|S_x(F)| = 2$. Then $|P_x(\pi, \alpha)|$ should still be less than d , on expectation. Indeed, for any unsatisfiable color c , there is a certain chance that $P^{(w)}$ “catches” it and thus excludes it from $P_x(\pi, \alpha)$. This probability is not formalized in the definition of p , the error rate, but we could come up with some bound for it in case of the heuristic $P^{(w)}$. However, there are some additional difficulties: if $|S_x(F)| \geq 2$, then [Lemma 4.4](#) does not apply anymore, and $R|\alpha$ quite possibly does *not* delay x . Thus, $\mathbb{E}_{\pi \sim R|\alpha}[\log |P_x(\pi, \alpha)|]$ might be significantly larger than under uniform π .

The other point in the proof where we possibly give away a lot is in the proof of [Proposition 5.19](#). We argue that “every liquid variable has at least 2 possible colors”. This might well be tight: $\text{sat}(F)$ might well be contained in the set $\{1, 2\}^n$, for example. However, this is an extreme case, and the pessimistic scenario that $|S_x(F)| \leq 2$ for all (or even most) x might turn out to be beneficial at some other point in the analysis.

Finally, we would like to prove [Theorem 1.13](#) in a more abstract way; by “more abstract” we mean a proof that uses only a few key properties of the proof heuristic and, unlike [Scheder \(2019\)](#) does not rely on the details of the actual heuristic being used—in that case, $P^{(w)}$. Also, it is quite unsatisfactory that [Scheder \(2019\)](#) fails for $k = 3, 4$.

Acknowledgements

Dominik Scheder gratefully acknowledges support by the National Natural Science Foundation of China under grant 61502300.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Funding

Open Access funding enabled and organized by Projekt DEAL.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, <http://creativecommons.org/licenses/by/4.0/>.

References

- SHITENG CHEN, DOMINIK SCHEDER, NAVID TALEBANFARD & BANGSHENG TANG (2013). Exponential lower bounds for the PPSZ k -SAT algorithm. In *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2013, New Orleans, Louisiana, USA, January 6-8, 2013*, SANJEEV KHANNA, editor, 1253–1263. SIAM. ISBN 978-1-61197-251-1. URL <http://dx.doi.org/10.1137/1.9781611973105.91>.
- THOMAS DUEHOLM HANSEN, HAIM KAPLAN, OR ZAMIR & URI ZWICK (2019). Faster k -SAT algorithms using biased-PPSZ. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing, STOC 2019, Phoenix, AZ, USA, June 23-26, 2019*, MOSES CHARIKAR & EDITH COHEN, editors, 578–589. ACM. URL <https://doi.org/10.1145/3313276.3316359>.
- TIMON HERTLI (2011). 3-SAT faster and simpler—Unique-SAT bounds for PPSZ hold in general. In *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science—FOCS 2011*, 277–284. IEEE Computer Soc., Los Alamitos, CA. URL <http://dx.doi.org/10.1109/FOCS.2011.22>.

TIMON HERTLI (2014). Breaking the PPSZ barrier for Unique 3-SAT. In *Automata, Languages, and Programming - 41st International Colloquium, ICALP 2014, Copenhagen, Denmark, July 8-11, 2014, Proceedings, Part I*, JAVIER ESPARZA, PIERRE FRAIGNIAUD, THORE HUSFELDT & ELIAS KOUTSOUPIS, editors, volume 8572 of *Lecture Notes in Computer Science*, 600–611. Springer. ISBN 978-3-662-43947-0. URL http://dx.doi.org/10.1007/978-3-662-43948-7_50.

TIMON HERTLI, ISABELLE HURBAIN, SEBASTIAN MILLIUS, ROBIN A. MOSER, DOMINIK SCHEDER & MAY SZEDLÁK (2016). The PPSZ algorithm for constraint satisfaction problems on more than two colors. In *Principles and Practice of Constraint Programming - 22nd International Conference, CP 2016, Toulouse, France, September 5-9, 2016, Proceedings*, MICHEL RUEHER, editor, volume 9892 of *Lecture Notes in Computer Science*, 421–437. Springer. URL https://doi.org/10.1007/978-3-319-44953-1_27.

TIMON HERTLI, ROBIN A. MOSER & DOMINIK SCHEDER (2011). Improving PPSZ for 3-SAT using Critical Variables. In *Proceedings of STACS*, 237–248.

RUSSELL IMPAGLIAZZO & RAMAMOHAN PATURI (2001). On the complexity of k -SAT. *J. Comput. Syst. Sci.* **62**(2), 367–375. URL <http://dx.doi.org/10.1006/jcss.2000.1727>.

KAZUO IWAMA, KAZUHISA SETO, TADASHI TAKAI & SUGURU TAMAKI (2010). Improved randomized algorithms for 3-SAT. In *Algorithms and Computation*, volume 6506 of *Lecture Notes in Comput. Sci.*, 73–84. Springer Berlin / Heidelberg.

KAZUO IWAMA & SUGURU TAMAKI (2004). Improved upper bounds for 3-SAT. In *Proceedings of the Fifteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, 328–329 (electronic). ACM, New York.

SHIBO LI & DOMINIK SCHEDER (2021). Impatient PPSZ - a faster algorithm for CSP. In *32nd International Symposium on Algorithms and Computation, ISAAC 2021, December 6-8, 2021, Fukuoka, Japan*, HEE-KAP AHN & KUNIHICO SADAKANE, editors, volume 212 of *LIPIcs*, 33:1–33:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik. URL <https://doi.org/10.4230/LIPIcs.ISAAC.2021.33>.

RAMAMOHAN PATURI, PAVEL PUDLÁK, MICHAEL E. SAKS & FRANCIS ZANE (2005). An improved exponential-time algorithm for k -SAT. *J.*

ACM **52**(3), 337–364 (electronic). ISSN 0004-5411. URL <http://dx.doi.org/10.1145/1066100.1066101>.

RAMAMOCHAN PATURI, PAVEL PUDLÁK & FRANCIS ZANE (1999). Satisfiability coding lemma. *Chicago J. Theoret. Comput. Sci.* Article 11, 19 pp. (electronic). ISSN 1073-0486.

PAVEL PUDLÁK, DOMINIK SCHEDER & NAVID TALEBANFARD (2017). Tighter hard instances for PPSZ. In *44th International Colloquium on Automata, Languages, and Programming, ICALP 2017, July 10-14, 2017, Warsaw, Poland*, 85:1–85:13.

TONG QIN & OSAMU WATANABE (2020). An improvement of the algorithm of Hertli for the unique 3SAT problem. *Theor. Comput. Sci.* **806**, 70–80. URL <https://doi.org/10.1016/j.tcs.2018.11.023>.

DANIEL ROLF (2006). Improved bound for the PPSZ/Schöningg-algorithm for 3-SAT. *Journal on Satisfiability, Boolean Modeling and Computation* **1**, 111–122.

DOMINIK SCHEDER (2019). PPSZ for $k \geq 5$: more is better. *TOCT* **11**(4), 25:1–25:22. URL <https://doi.org/10.1145/3349613>.

DOMINIK SCHEDER (2021). PPSZ is better than you think. In *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*, 205–216. IEEE. URL <https://doi.org/10.1109/FOCS52979.2021.00028>.

DOMINIK SCHEDER & JOHN P. STEINBERGER (2017). PPSZ for general k -SAT - making Hertli’s analysis simpler and 3-SAT faster. In *32nd Computational Complexity Conference, CCC 2017, July 6-9, 2017, Riga, Latvia*, RYAN O’DONNELL, editor, volume 79 of *LIPICs*, 9:1–9:15. Schloss Dagstuhl - Leibniz-Zentrum fuer Informatik. ISBN 978-3-95977-040-8. URL <https://doi.org/10.4230/LIPICs.CCC.2017.9>.

DOMINIK SCHEDER & NAVID TALEBANFARD (2020). Super strong ETH is true for PPSZ with small resolution width. In *35th Computational Complexity Conference, CCC 2020, July 28-31, 2020, Saarbrücken, Germany (Virtual Conference)*, SHUBHANGI SARAF, editor, volume 169 of *LIPICs*, 3:1–3:12. Schloss Dagstuhl - Leibniz-Zentrum für Informatik. URL <https://doi.org/10.4230/LIPICs.CCC.2020.3>.

UWE SCHÖNING (1999). A probabilistic algorithm for k -SAT and constraint satisfaction problems. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science*, 410–414. IEEE Computer Society, Los Alamitos, CA. URL <http://dx.doi.org/10.1109/SFFCS.1999.814612>.

NIKHIL VYAS & R. RYAN WILLIAMS (2019). On super strong ETH. In *Theory and Applications of Satisfiability Testing - SAT 2019 - 22nd International Conference, SAT 2019, Lisbon, Portugal, July 9-12, 2019, Proceedings*, MIKOLÁS JANOTA & INÊS LYNCE, editors, volume 11628 of *Lecture Notes in Computer Science*, 406–423. Springer. URL https://doi.org/10.1007/978-3-030-24258-9_28.

OR ZAMIR (2021). Faster algorithm for unique $(k, 2)$ -CSP. *CoRR abs/2110.03122*. URL <https://arxiv.org/abs/2110.03122>.

Manuscript received 1 August 2022

DOMINIK SCHEDER
TU Chemnitz, Chemnitz,
Germany
dominik.scheder@gmail.com

JOHN STEINBERGER
Auki Labs, Kowloon Bay,
Hong Kong
jpsteinb@gmail.com